

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
Рязанский государственный радиотехнический университет
имени В.Ф. Уткина

Кафедра «ВПМ»

К защите
«__» _____ 2026 г.
_____ *подпись*

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

по дисциплине

«Клиент-серверные приложения»

по теме

«Разработка информационной системы с клиент-серверной архитектурой»

Выполнил

ст. гр. 243

Вербицкая Ирина Сергеевна

_____ *дата сдачи на проверку, подпись*

Проверил

доцент, к.т.н.

Макаров Николай Петрович

оценка

дата защиты, подпись

ЗАДАНИЕ

на курсовой проект по дисциплине

“Клиент-серверные приложения баз данных”

Исполнители: Вербицкая Ирина Сергеевна

Тема: «Разработка информационной системы с клиент-серверной архитектурой»

Вариант задания: разработать клиент-серверное приложение для управления сетью кондитерских быстрого обслуживания, которая позволяла бы посетителям заказывать позиции из меню, а кассирам - вести следить за выполнением заказов.

Требования к используемому программному обеспечению:

- СУБД: PostgreSQL;
- язык программирования: Java;
- CASE средства проектирования баз данных: PowerDesigner 15, drawio.

Общие требования к проекту:

Разработать информационную систему (ИС) с клиент-серверной архитектурой, используя двухзвенную модель (сервер баз данных).

Провести логическое и физическое проектирование базы данных (БД) используя CASE инструментарий. Проверить все таблицы БД на соответствие нормальной форме Бойса-Кодда. Предусмотреть контроль целостности данных на уровне БД: ограничения, триггеры, хранимые процедуры и т.д. Сформировать SQL-скрипт для создания БД в одном из вариантов: независимый sql-скрипт, liquibase-проект, maven- или java-проект с liquibase. В целом, в БД должно быть не менее 10 таблиц.

Клиентская часть ИС должна реализовывать необходимую пользователю функциональность. Предусмотреть возможность печати необходимых пользователю отчетов.

Разработать результирующие дистрибутивы для независимой установки/запуска БД и клиентского приложения.

К защите представить:

- работоспособную программу и БД, установленные на аудиторном компьютере (допускается демонстрация работы проекта на ноутбуке);
- пояснительную записку в распечатанном виде (исходные тексты программ не печатать);
- CD-диск, содержащий пояснительную записку, дистрибутивы для ИС, исходные тексты программ, SQL-скрипт для создания БД, файлы БД, сценарий инсталляции/запуска клиентского приложения.

Требования к содержанию пояснительной записки:

Введение.

1. Анализ задачи.

1.1. Анализ предметной области, выявление необходимой пользователю функциональности.

1.2. Разработка общей архитектуры ИС.

2. Разработка серверной части информационной системы.

2.1. Выявление сущностей, их атрибутов и ключей.

2.2. Выявление связей.

2.3. Построение ER-диаграммы.

2.4. Переход от ER-диаграммы к предварительным отношениям.

2.5. Заполнение предварительных отношений атрибутами.

2.6. Проверка предварительных отношений на соответствие нормальным формам.

2.7. Построение схемы данных.

- 2.8. Задание частных ограничений целостности данных.
- 2.9. Разработка триггеров, хранимых процедур и др.
- 3. Разработка клиентской части информационной системы.
 - 3.1. Организация взаимодействия клиентской программы с БД.
 - 3.2. Разработка форм.
 - 3.3. Разработка отчетов
 - 3.4. Разработка сценария инсталляции/запуска клиентской программы.
 - 3.5. Руководство пользователя.

Заключение.

Список используемой литературы.

Приложение 1: SQL-скрипт для создания БД.

Приложение 2: исходный текст клиентской программы.

Приложение 3: сценарий инсталляции/запуска программы.

Примечания:

- 1. В п.2.3 включить изображение ER-диаграммы.
- 2. В п.2.6 включить изображения диаграмм функциональных зависимостей атрибутов для всех таблиц.
- 3. В п.2.7 включить изображение схемы данных.
- 4. Приложения 1–3 не распечатывать, включив их лишь в электронную версию пояснительной записки.

Дата выдачи задания: «__» _____ 2026 г.

Срок сдачи проекта: __ ____ 2026 г.

Подписи исполнителей:

Подпись руководителя:

Оглавление

Введение.....	6
1 Анализ задачи	7
1.1 Анализ предметной области, выявление необходимой пользователю функциональности.	7
1.2 Разработка общей структуры ИС	10
2 Разработка серверной части информационной системы.....	11
2.1 Выявление сущностей, их атрибутов и ключей.....	11
2.2 Выявление связей.....	11
2.3 Построение ER-диаграммы.....	13
2.4 Переход от ER-диаграммы к предварительным отношениям.	15
2.5 Заполнение предварительных отношений атрибутами.	19
2.6 Проверка предварительных отношений на соответствие нормальным формам.....	23
2.7 Построение схемы данных.....	27
2.8 Задание частных ограничений целостности данных.....	30
2.9 Разработка триггеров и хранимых процедур	33
3 Разработка клиентской части информационной системы	44
3.1 Организация взаимодействия клиентской программы с БД ...	44
3.2 Разработка форм.....	44
3.3 Разработка отчётов.....	59
3.4 Разработка сценария инсталляции клиентской программы.....	63
3.5 Руководство пользователя	63
Заключение	68

Список используемой литературы	70
--------------------------------------	----

Введение

Сеть кондитерских быстрого обслуживания «Последний пончик» представляет собой современную систему общественного питания, ориентированную на быстрое и удобное обслуживание клиентов. В условиях высокой конкуренции и растущих требований потребителей к качеству сервиса особую роль играет автоматизация процессов оформления, обработки и выдачи заказов. Использование информационных технологий позволяет ускорить обслуживание посетителей, повысить точность учета данных и обеспечить эффективное взаимодействие между сотрудниками и клиентами.

Работа сети кондитерских связана с большим количеством ежедневных операций: оформлением заказов различными способами, учетом ассортимента изделий, контролем выполнения заказов, хранением информации о клиентах и сотрудниках, а также управлением ценами и меню. Без использования специализированной информационной системы управление подобной сетью становится сложной и трудоемкой задачей. Хранение информации в бумажном виде или в разрозненных электронных таблицах приводит к увеличению времени обработки заказов, усложняет контроль за выполнением операций и повышает вероятность возникновения ошибок. Особенно важным становится хранение истории выполнения заказов, отслеживание изменения цен на изделия и поддержка актуального меню для каждого ресторана сети.

Внедрение клиент-серверной информационной системы позволит автоматизировать основные бизнес-процессы сети. Все данные о ресторанах, заказах, сотрудниках, меню, ингредиентах и изменении цен будут храниться в единой базе данных. Это обеспечит быстрый доступ к необходимой информации, повысит качество обслуживания клиентов, упростит контроль над работой заведений и позволит эффективно анализировать деятельность сети в целом.

1 Анализ задачи

1.1 Анализ предметной области, выявление необходимой пользователю функциональности.

Сеть кондитерских быстрого обслуживания, объединенных определенным брендом, пусть это будет «Последний пончик», представляет собой несколько заведений-кондитерских быстрого обслуживания в одном городе. *Клиенты* приходят в *ресторан* по одному из адресов, где могут оформить заказ *3 способами*: на кассе ресторана, через терминал самообслуживания, в приложении кондитерской.

При покупке любым из способов заказ можно получить у стойки выдачи по чеку. При оформлении через кассу заказ можно получить только по чеку. При оформлении через терминал у клиента также есть возможность взять локатор, к которому официант принесет заказ. При оформлении через приложение у пользователя есть возможность указать номер стола, к которому принесут заказ.

Меню доступных к заказу *изделий* может отличаться у разных ресторанов. Все *стадии выполнения заказа* фиксируются в системе. Система должна помнить завершенные заказы, историю их выполнения, историю смены цен на изделия. Изделия разделены в меню по *категориям*. Для каждого изделия доступен список его *ингредиентов*.

Предметной областью данной курсовой работы является управление деятельностью сетью кондитерских.

Данный проект посвящен разработке клиент-серверного приложения для управления сетью кондитерских, которое позволит автоматизировать ключевые бизнес-процессы и предоставить посетителям и сотрудникам функциональный инструмент для работы с базой данных.

Ключевые участники: клиенты, кассиры, администраторы.

Анализ требований пользователей.

Требования клиентов:

Просмотр меню ресторана по категориям.

Просмотр основной информации об изделиях – названия, стоимости, описания, ингредиентов, калорийности.

Добавление и удаление изделий в заказ, изменение количества изделий в заказе.

Оформление заказа через приложение или терминал самообслуживания.

При заказе через приложение – просмотр истории своих заказов.

При заказе через приложение – выбор предприятия (ресторана) где осуществляется заказ.

Возможность указать номер столика при заказе через приложение.

Возможность указать номер локатора при заказе через терминал.

Отслеживание статуса текущих заказов через приложение.

При заказе через приложение – возможность входа в свой аккаунт по номеру телефона.

Требования кассиров:

Просмотр меню ресторана по категориям.

Просмотр основной информации об изделиях – названия, стоимости, описания, ингредиентов, калорийности.

Добавление и удаление изделий в заказ, изменение количества изделий в заказе.

Возможность указать номер телефона клиента при оформлении заказа.

Оформление заказа для клиента на кассе.

Просмотр истории завершенных заказов.

Просмотр текущих заказов, изменение их статуса.

Возможность выхода из своего аккаунта и входа.

Требования администраторов:

Добавление новой, либо изменение основной существующей информации о кассирах, изделиях, меню ресторанов, категориях изделий, ингредиентах изделий, ценах на изделия.

Удаление информации о категориях и ингредиентах изделий.

Необходимая функциональность информационной системы:

Авторизация пользователей системы.

Добавление клиентами и кассирами изделий в корзину, настройка количества изделий в корзине.

Оформление заказов клиентами через приложение или терминал самообслуживания.

Оформление заказов кассирами на кассе ресторана.

Обновление кассирами статуса приготовления заказа.

Добавление и изменение изделий администраторами.

Добавление, изменение и удаление категорий администраторами.

Изменение состава изделий администраторами.

Добавление, изменение и удаление ингредиентов администраторами.

Изменение меню ресторанов администраторами.

Изменение цен на изделия администраторами.

Добавление и изменение кассиров администраторами.

Получение администраторами отчета о выручке ресторанов.

Получение администраторами отчета об изменении цен на изделия.

Таким образом, информационная система должна обеспечивать комплексное управление деятельностью сетью кондитерских, автоматизируя ключевые бизнес-процессы и предоставляя удобный интерфейс для всех участников процесса. Это позволит повысить эффективность работы сети, улучшить качество обслуживания клиентов и обеспечить более тесное взаимодействие между различными участниками.

1.2 Разработка общей структуры ИС

Необходимо выбрать архитектуру проектируемой системы.

Для данной системы наиболее подходящим выбором будет двухуровневая архитектура клиент/сервер. Модель архитектуры представлена на рисунке 1.

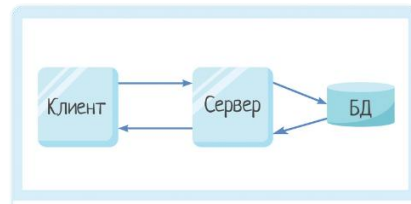


Рисунок 1 – Двухуровневая архитектура клиент/сервер

В этой модели бизнес-логика разделена между клиентом и сервером. На сервере бизнес-логика реализована в виде хранимых процедур, которые хранятся в БД и управляются непосредственно СУБД. Клиентское приложение обращается к серверу с командой запуска хранимой процедуры, а сервер выполняет эту процедуру и регистрирует все изменения в БД, которые в ней предусмотрены. Сервер возвращает клиенту данные, релевантные его запросу, которые требуются клиенту либо для вывода на экран, либо для выполнения части бизнес-логики, которая расположена на клиенте.

Авторизация с помощью номера телефона. Пользователю доступно три варианта входа в приложение: клиент, кассир, администратор.

Главное окно клиента. Оно будет включать в себя меню ресторана с изделиями доступными для заказа.

Главное окно кассира. Оно будет включать в себя список заказов, а также кнопку для оформления новых.

Главное окно администратора. Будет представлена возможность по нажатию на кнопки переходить на окна управления кассирами, меню, изделиями, ингредиентами, категориями изделий. Также администратор может создавать отчеты о заказах ресторанов, о наиболее популярных изделиях.

2 Разработка серверной части информационной системы.

2.1 Выявление сущностей, их атрибутов и ключей.

В процессе разработки серверной части информационной системы удалось определить сущности, их атрибуты и ключи:

Клиент (КодКлиента, Имя, НомерТелефона). Посетитель кондитерской.

Ресторан (НомерРесторана, Адрес). Одно из заведений сети кондитерских.

Кассир (КодКассира, Паспорт, НомерРесторана, Фамилия, Имя, НомерТелефона, ИНН). Кассир, работающий в заведении кондитерской. Оформляет заказы клиентов и принимает оплату заказов.

Изделие (КодИзделия, НомерКатегории, Название, Описание, ПищеваяЦенность, ТекущаяСтоимость). Блюдо, доступное к заказу в ресторане.

КатегорияИзделия (НомерКатегории, Название, Описание). Группа изделий, объединенных по какому-либо признаку.

Ингредиент (КодИнгредиента, Название, Описание, Аллергенность). Продукт, входящий в состав изделия по его рецепту.

СтатусЗаказа (КодСтатуса, Название, Описание). Состояние заказа во времени, например: «создан», «отменен», «готовится», «готов», «завершен».

2.2 Выявление связей.

В процессе разработки серверной части информационной системы удалось выявить следующие связи между сущностями:

Кассир работает в ресторане. Показывает, какие кассиры работают в ресторане. Один ко многим: в каждом ресторане работает много кассиров, каждый кассир может работать только в одном ресторане. Кассир обязателен в связи: не может быть кассиров, не работающих ни в одном из ресторанов.

Сущность изделие обладает важной особенностью. Исходя из требований предметной области необходимо хранить историю изменения

цены на изделие. Для выполнения этого требования введена простейшую сущность *Время*, являющуюся атрибутом, в дальнейшем она не будет представлена в виде отдельной таблицы.

Изделие имеет цену в момент. Показывает историю смены цен на изделие. Многие ко многим: в каждый момент времени может смениться цена сразу на несколько изделий, каждое изделие может менять цену во множество моментов времени.

Изделие имеет категорию. Показывает категории, в которые входит то или иное изделие. Один ко многим: каждое изделие может входить только в одну категорию, а в одну категорию может входить множество изделий. Изделие обязательно в связи: не должно быть изделий, не входящих ни в одну категорию.

Изделие имеет в своем составе ингредиенты. Показывает состав изделия. Многие ко многим: каждое изделие может иметь в составе множество ингредиентов, а каждый ингредиент может входить в состав множества изделий.

Изделия доступны в ресторане к заказу. Показывает меню ресторана, так как оно может отличаться от ресторана к ресторану. Многие ко многим: в каждом ресторане доступно к заказу множество изделий, каждое изделие доступно к заказу во многих ресторанах.

Клиент с помощью кассира покупает в ресторане изделия, которые по мере выполнения изменяют свое состояние готовности (статус заказа). Иллюстрирует основную в выбранной предметной области связь покупки клиентом изделий; эта сложная связь задействует большое количество сущностей. Для всех сущностей это связь многие ко многим.

- Один и тот же набор из кассира, ресторана, изделия со статусом готовности может повторяться (быть заказанным) у многих клиентов. Например, если две одинаковых покупки совершат два разных клиента

– перечисленные сущности для этой покупки могут совпасть. Клиент обязателен в связи, так как играет ключевую роль в процессе покупки, покупка не может быть осуществлена «ником».

- Один и тот же набор из кассира, ресторана, клиента и изделия может соответствовать нескольким статусам заказа. Например, если клиент купит определенный набор изделий, связанный с перечисленными сущностями – состояние их готовности будет изменяться по мере их приготовления, то есть будет записано множество состояний.
- Один и тот же набор из кассира, ресторана, клиента и статуса заказа может соответствовать многим изделиям, купленным в этот раз. Например, когда покупатель заказывает множество изделий у кассира – прочие сущности совпадают, но изделий при этом множество. Изделие обязательно в связи – клиент не может «купить» «не покупая ничего», то есть он обязательно должен купить хотя бы одно изделие для формирования этой связи.
- Один и тот же набор из ресторана, клиента, изделия со статусом готовности может быть оформлен разными кассирами. Например, когда клиент дважды заказывает одни и те же изделия у разных кассиров – прочие сущности могут совпадать. Кассир обязателен в связи – за ходом приготовления заказа обязательно следит человек.
- Каждому набору из кассира, клиента, изделия со статусом готовности может соответствовать только один ресторан, поскольку каждый кассир может работать только в одном ресторане. Ресторан обязателен в связи – покупка обязательно происходит в одном из заведений сети кондитерских, она не может происходить «нигде».

2.3 Построение ER-диаграммы.

После выявления сущностей и определив связи между ними, построим концептуальную модель данных для предметной области «Сеть

кондитерских». Результаты представлены на рисунках: модель из draw.io (рисунок 2) и модель из PowerDesigner (рисунок 3).

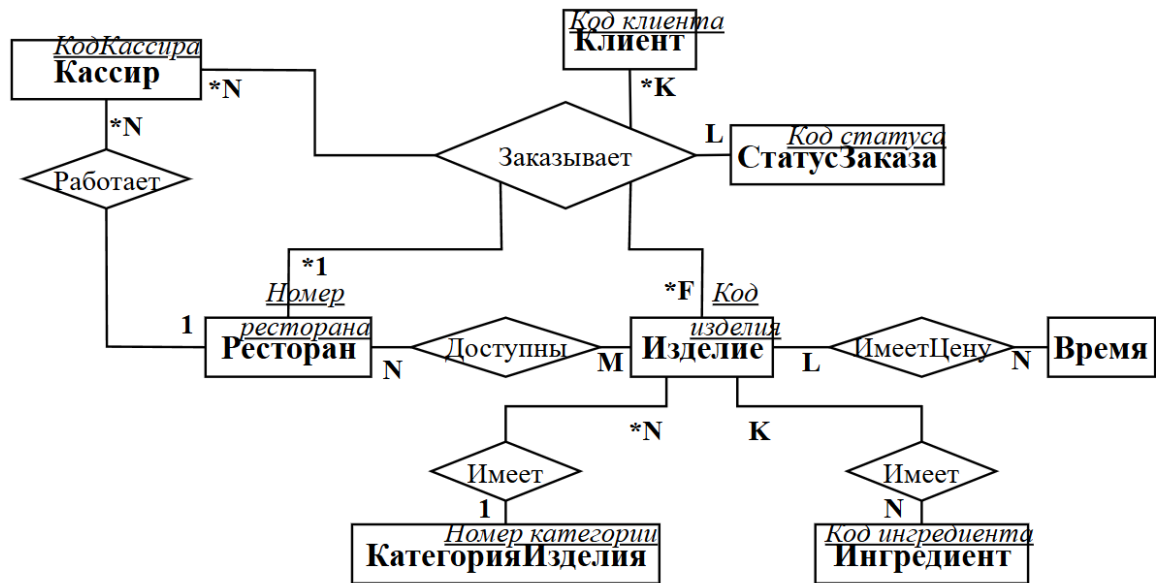


Рисунок 2 – Концептуальная модель

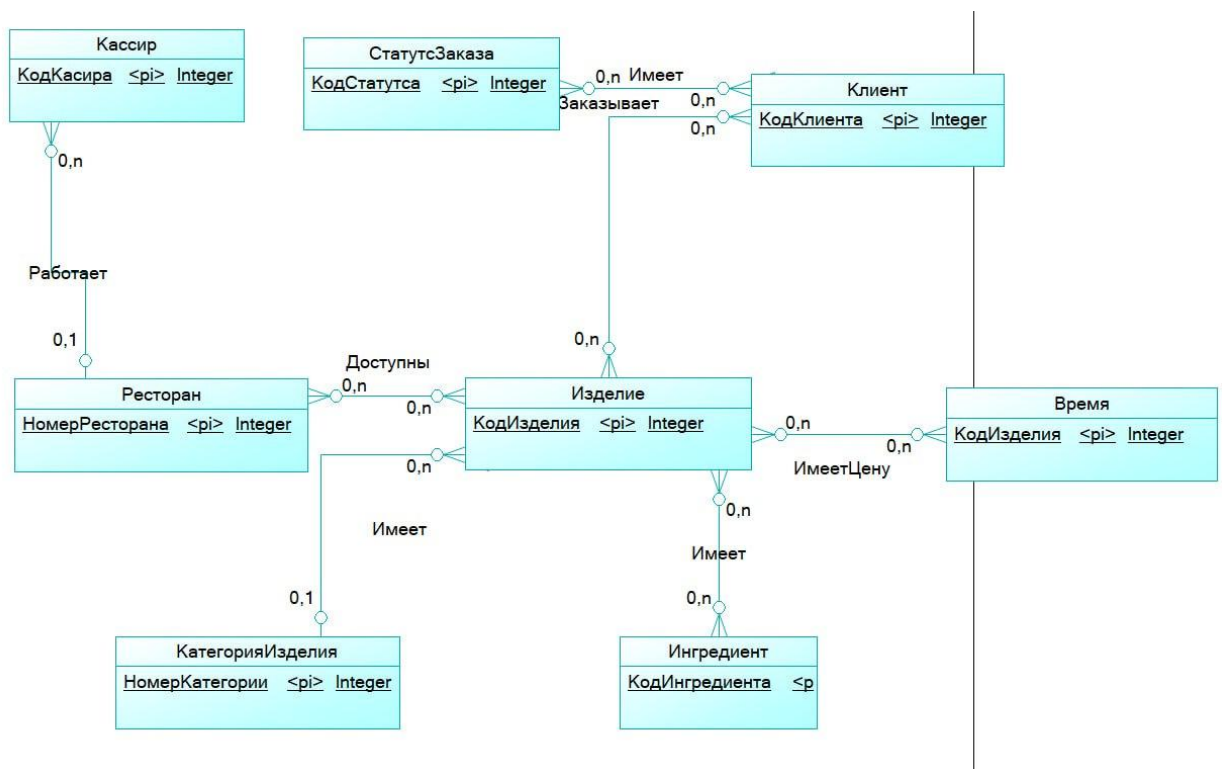


Рисунок 3 – ER диаграмма в PowerDesigner

2.4 Переход от ER-диаграммы к предварительным отношениям.

Для перехода от концептуальной модели базы данных, к логической модели базы данных, основанной на реляционной модели, необходимо избавиться от связей «Многие ко многим». Для этого необходимо преобразовать связи «Многие ко многим» в слабые сущности со связями «Один ко многим».

Выделим слабую сущность:

Заказ (НомерЗаказа, КодКлиента, КодКассира, НомерРесторана, Стоимость, НомерСтолика, НомерЛокатора, ВремяОформления). Документ, отражающий информацию о покупке клиентом определенного набора изделий в кондитерской.

А также связи:

Заказ относится к ресторану. Показывает ресторан, в котором был сделан заказ. Один ко многим: каждый заказ можно сделать только в одном ресторане, но в каждом ресторане можно сделать множество заказов. Заказ обязателен в связи: не может быть заказа, для которого не определен ресторан.

Кассир оформляет заказ. Показывает кассира, оформившего заказ. Один ко многим: каждый кассир может оформить много заказов, но у каждого заказа может быть только один кассир. Заказ обязателен в связи: не может быть заказа, за которым не следит кассир.

Заказ имеет статус готовности (статус заказа). Отражает ход смены статусов заказа, ход приготовления заказа. Многие ко многим: каждому статусу может соответствовать множество заказов, каждому заказу может соответствовать множество статусов.

Заказ содержит изделия. Отражает список товаров в заказе. Многие ко многим: в каждом заказе может быть много изделий, каждое изделие может быть заказано множество раз и включено во множество заказов. Заказ

обязателен в связи: не может быть пустого заказа «ни на что» - не включающего изделия.

Клиент делает заказ. Показывает клиента, оплатившего заказ. Один ко многим: каждому заказу соответствует лишь один клиент, но каждый клиент может много раз заказать что-то. Заказ обязателен в связи: не может быть заказа без заказчика (клиента).

В результате получаем промежуточную логическую модель (рисунок 4).

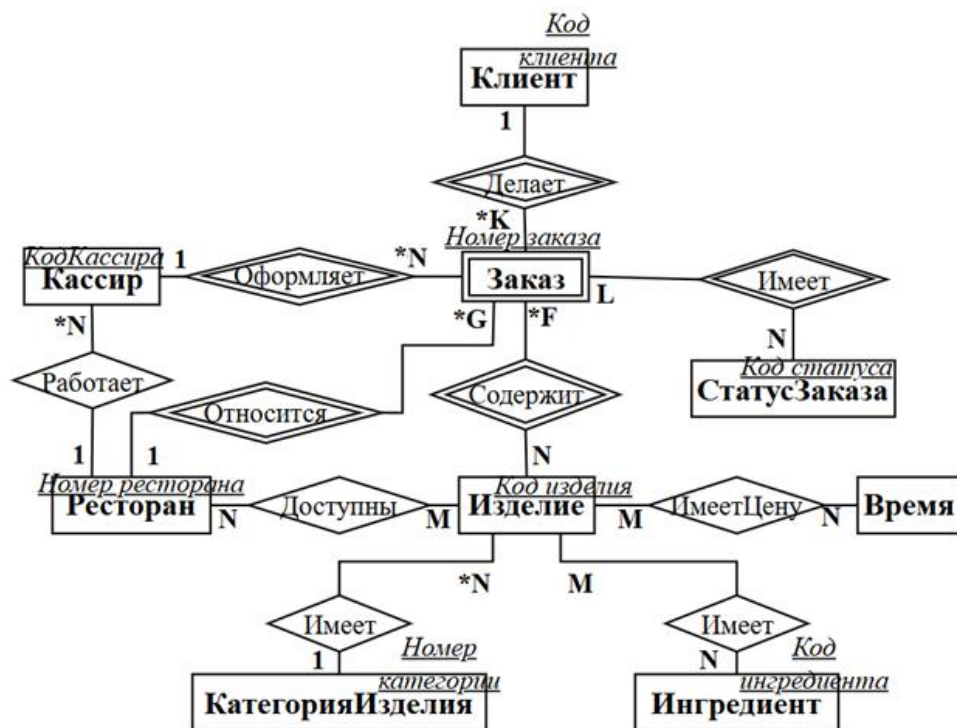


Рисунок 4 – Промежуточная логическая модель в drawio

В результате преобразования у нас образовались новые бинарные связи «Многие ко многим». Преобразуем каждую из них через формирование слабых сущностей:

СменаСтатусовЗаказа (НомерЗаказа, ДатаВремя, КодСтатуса).
Представляет собой историю выполнения заказа.

МенюРесторана (НомерРесторана, КодИзделия). Представляет собой набор изделий, доступных в ресторанах к заказу.

СменаЦенНаИзделия (КодИзделия, ДатаВремя, Цена). Представляет собой историю изменения стоимости изделий.

СоставИзделия (КодИзделия, КодИнгредиента). Показывает ингредиенты, входящие в состав изделия.

СоставЗаказа (НомерЗаказа, КодИзделия, Количество). Показывает изделия, купленные в заказе, а также их количество.

А также связей **включает**. Связь **включает** в данном случае универсальна. Это всегда связь один ко многим, когда сущность «один» может включать множество сущностей «многие», но сущности «многие» соответствует лишь одна сущность «один». «Многие» обязательна в связи, поскольку не может существовать без обязательного указания «один».

После заполнения полей и нормализации отношений построена реляционная логическая модель базы данных. Результаты представлены на рисунках: модель из draw.io (рисунок 5) и модель из PowerDesigner (рисунок 6).

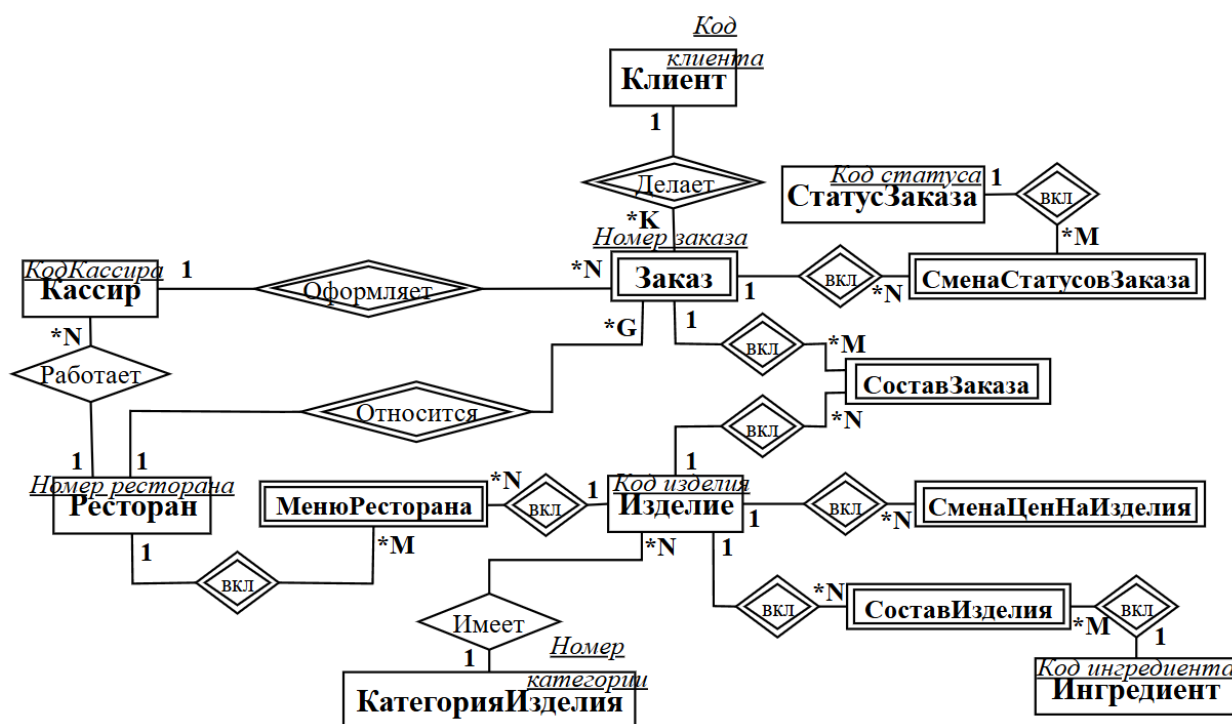


Рисунок 5 – Логическая модель в drawio

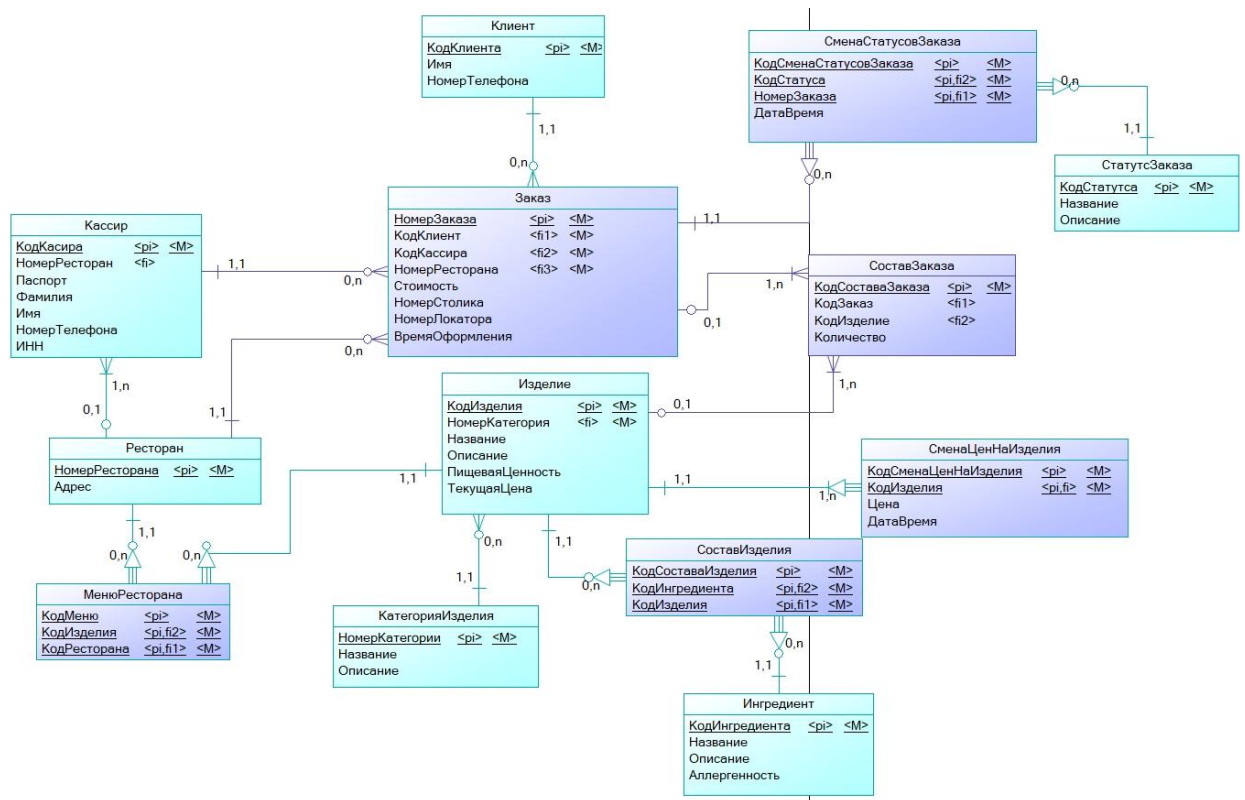


Рисунок 6 – Логическая модель в PowerDesigner

Таким образом, полный набор сущностей имеет вид:

Клиент (КодКлиента, Имя, НомерТелефона).

Ресторан (НомерРесторана, Адрес).

Кассир (КодКассира, Паспорт, НомерРесторана, Фамилия, Имя, НомерТелефона, ИНН).

Издeлие (КодИздeлия, НомерКатегории, Название, Описание, ПищеваяЦенность, ТекущаяСтоимость).

КатегорияИздeлия (НомерКатегории, Название, Описание).

Ингредиент (КодИнгредиента, Название, Описание, Аллергенность).

СтатусЗаказа (КодСтатуса, Название, Описание).

Заказ (НомерЗаказа, КодКлиента, КодКассира, НомерРесторана, Стоимость, НомерСтолика, НомерЛокатора, ВремяОформления).

СменаСтатусовЗаказа (НомерЗаказа, ДатаВремя, КодСтатуса)

МенюРесторана (НомерРесторана, КодИздeлия)

СменаЦенНаИздeлия (КодИздeлия, ДатаВремя, Цена)

СоставИзделия (КодИзделия, КодИнгредиента)

СоставЗаказа (НомерЗаказа, КодИзделия, Количество)

Для слабых сущностей введем дополнительный атрибут (первичный ключ) код:

СменаСтатусовЗаказа (Код, НомерЗаказа, ДатаВремя, КодСтатуса)

МенюРесторана (Код, НомерРесторана, КодИзделия)

СменаЦенНаИзделия (Код, КодИзделия, ДатаВремя, Цена)

СоставИзделия (Код, КодИзделия, КодИнгредиента)

СоставЗаказа (Код, НомерЗаказа, КодИзделия, Количество)

2.5 Заполнение предварительных отношений атрибутами.

Таблица 1 – Атрибуты таблицы Клиент

Поле	Тип	Ограничения
КодКлиента	INT	PRIMARY KEY
Имя	NVARCHAR(15)	NOT NULL
НомерТелефона	CHAR(11)	NOT NULL, CHECK (НомерТелефона NOT LIKE '%[^0-9]%' AND LENGTH(НомерТелефона)=11)

Таблица 2 – Атрибуты таблицы Ресторан

Поле	Тип	Ограничения
НомерРесторана	INT	PRIMARY KEY
Адрес	NVARCHAR(50)	NOT NULL

Таблица 3 – Атрибуты таблицы Кассир

Поле	Тип	Ограничения
КодКассира	INT	PRIMARY KEY

Паспорт	CHAR(10)	NOT NULL, CHECK (Паспорт NOT LIKE '%[^0-9]%' AND LENGTH(Паспорт)=10)
НомерРесторана	INT	FOREIGN KEY
Фамилия	NVARCHAR(20)	NOT NULL
Имя	NVARCHAR(15)	NOT NULL
НомерТелефона	CHAR(11)	NOT NULL, CHECK (НомерТелефона NOT LIKE '%[^0-9]%' AND LENGTH(НомерТелефона)=11)
ИНН	CHAR(12)	NOT NULL, CHECK (ИНН NOT LIKE '%[^0-9]%' AND LENGTH(ИНН)=12)

Таблица 4 – Атрибуты таблицы Изделие

Поле	Тип	Ограничения
КодИзделия	INT	PRIMARY KEY
НомерКатегории	INT	FOREIGN KEY, NOT NULL
Название	NVARCHAR(30)	NOT NULL, UNIQUE
Описание	NVARCHAR(200)	
ПищеваяЦенность	INT	NOT NULL
ТекущаяСтоимость	FLOAT	

Таблица 5 – Атрибуты таблицы КатегорияИзделия

Поле	Тип	Ограничения
НомерКатегории	INT	PRIMARY KEY
Название	NVARCHAR(15)	NOT NULL, UNIQUE
Описание	NVARCHAR(50)	

Таблица 6 – Атрибуты таблицы Ингредиент

Поле	Тип	Ограничения
КодИнгредиента	INT	PRIMARY KEY
Название	NVARCHAR(15)	NOT NULL, UNIQUE
Описание	NVARCHAR(50)	
Аллергенность	BOOLEAN	NOT NULL, DEFAULT FALSE

Таблица 7 – Атрибуты таблицы СтатусЗаказа

Поле	Тип	Ограничения
КодСтатуса	INT	PRIMARY KEY
Название	NVARCHAR(15)	NOT NULL, UNIQUE
Описание	NVARCHAR(50)	

Таблица 8 – Атрибуты таблицы Заказ

Поле	Тип	Ограничения
НомерЗаказа	INT	PRIMARY KEY
КодКлиента	INT	FOREIGN KEY, NOT NULL
КодКассира	INT	FOREIGN KEY, NOT NULL
НомерРесторана	INT	FOREIGN KEY, NOT NULL
Стоимость	FLOAT	NOT NULL
НомерСтолика	INT	
НомерЛокатора	INT	
ВремяОформления	TIMESTAMP	NOT NULL

Таблица 9 – Атрибуты таблицы СменаСтатусовЗаказа

Поле	Тип	Ограничения
Код	INT	PRIMARY KEY

НомерЗаказа	INT	FOREIGN KEY, NOT NULL
ДатаВремя	TIMESTAMP	NOT NULL
КодСтатуса	INT	FOREIGN KEY, NOT NULL

Таблица 10 – Атрибуты таблицы МенюРесторана

Поле	Тип	Ограничения
Код	INT	PRIMARY KEY
НомерРесторана	INT	FOREIGN KEY, NOT NULL
КодИзделия	INT	FOREIGN KEY, NOT NULL

Таблица 11 – Атрибуты таблицы СменаЦенНаИзделие

Поле	Тип	Ограничения
Код	INT	PRIMARY KEY
КодИзделия	INT	FOREIGN KEY, NOT NULL
ДатаВремя	TIMESTAMP	NOT NULL
Цена	FLOAT	NOT NULL

Таблица 12 – Атрибуты таблицы СоставИзделия

Поле	Тип	Ограничения
Код	INT	PRIMARY KEY
КодИнгредиента	INT	FOREIGN KEY, NOT NULL
КодИзделия	INT	FOREIGN KEY, NOT NULL

Таблица 13 – Атрибуты таблицы СоставЗаказа

Поле	Тип	Ограничения
------	-----	-------------

Код	INT	PRIMARY KEY
НомерЗаказа	INT	FOREIGN KEY, NOT NULL
КодИзделия	INT	FOREIGN KEY, NOT NULL
Количество	INT	NOT NULL, DEFAULT 1

2.6 Проверка предварительных отношений на соответствие нормальным формам.

Осуществим проверку соответствия предварительных отношений нормальным формам. Для оценки соответствия второй нормальной форме будем рассматривать не искусственно добавленные первичные ключи слабых сущностей – а их первоначальные составные.

Таблица Клиент:

1НФ: соответствует. Атрибуты содержат только атомарные значения, и каждая строка уникально идентифицируется по первичному ключу КодКлиента, отсутствуют повторяющиеся атрибуты.

2НФ: соответствует. Все неключевые атрибуты (Имя, НомерТелефона) функционально зависят от первичного ключа КодКлиента.

3НФ: соответствует. Транзитивные зависимости между неключевыми атрибутами отсутствуют.

Таблица Ресторан:

1НФ: соответствует. Все поля содержат атомарные значения, каждая запись уникально идентифицируется по ключу НомерРесторана, отсутствуют повторяющиеся атрибуты.

2НФ: соответствует. Все неключевые атрибуты (Адрес) функционально зависят от первичного ключа НомерРесторана.

ЗНФ: соответствует. Транзитивные зависимости между неключевыми атрибутами отсутствуют.

Таблица Кассир:

1НФ: соответствует. Все поля атомарны, строки уникально идентифицируются по КодКассира, отсутствуют повторяющиеся атрибуты.

2НФ: соответствует. Все неключевые атрибуты (Паспорт, НомерРесторана, Фамилия, Имя, НомерТелефона, ИНН) функционально зависят от первичного ключа КодКассира.

ЗНФ: соответствует. Транзитивные зависимости между неключевыми атрибутами отсутствуют.

Таблица КатегорияИзделия:

1НФ: соответствует. Все значения атомарны, строки идентифицируются по НомерКатегории, отсутствуют повторяющиеся атрибуты.

2НФ: соответствует. Все неключевые атрибуты (Название и Описание) функционально зависят от первичного ключа НомерКатегории.

ЗНФ: соответствует. Транзитивные зависимости между неключевыми атрибутами отсутствуют.

Таблица Изделие:

1НФ: соответствует. Таблица содержит только атомарные значения, каждая запись определяется ключом КодИзделия, отсутствуют повторяющиеся атрибуты, отсутствуют повторяющиеся атрибуты.

2НФ: соответствует. Все неключевые атрибуты (НомерКатегории, Название, Описание, ПищеваяЦенность, ТекущаяСтоимость) функционально зависят от первичного ключа КодИзделия.

ЗНФ: соответствует. Транзитивные зависимости между неключевыми атрибутами отсутствуют.

Таблица Ингредиент:

1НФ: соответствует. Поля содержат только атомарные значения, строки уникально идентифицируются по КодИнгредиента, отсутствуют повторяющиеся атрибуты.

2НФ: соответствует. Все неключевые атрибуты (Название, Описание, Аллергенность) функционально зависят от первичного ключа (КодИнгредиента).

3НФ: соответствует. Транзитивные зависимости между неключевыми атрибутами отсутствуют.

Таблица СтатусЗаказа:

1НФ: соответствует. Все поля атомарны, записи идентифицируются по КодСтатуса, отсутствуют повторяющиеся атрибуты.

2НФ: соответствует. Все неключевые атрибуты (Название и Описание) функционально зависят от первичного ключа КодСтатуса.

3НФ: соответствует. Транзитивные зависимости между неключевыми атрибутами отсутствуют.

Таблица Заказ:

1НФ: соответствует. Все поля содержат атомарные значения, каждая запись уникально идентифицируется по НомерЗаказа, отсутствуют повторяющиеся атрибуты.

2НФ: соответствует. Атрибуты Стоимость, НомерСтолика, НомерЛокатора, ВремяОформления функционально полностью зависят от составного ключа (КодКлиента, КодКассира, НомерРесторана, ВремяОформления).

3НФ: соответствует. Все неключевые атрибуты не зависят друг от друга (отсутствуют транзитивные зависимости).

Таблица СменаСтатусовЗаказа:

1НФ: соответствует. Все значения атомарны, отсутствуют повторяющиеся атрибуты. Все записи идентифицируются первичным ключом Код.

2НФ: соответствует. Атрибут КодСтатуса функционально полно зависит от составного ключа (НомерЗаказа, ДатаВремя).

3НФ: соответствует. Транзитивные зависимости отсутствуют.

Таблица МенюРесторана:

1НФ: соответствует. Таблица содержит только атомарные значения, отсутствуют повторяющиеся атрибуты. Все записи идентифицируются первичным ключом Код.

2НФ: соответствует. Таблица состоит только из ключевых атрибутов НомерРесторана и КодИзделия, неключевые атрибуты отсутствуют.

3НФ: соответствует. Транзитивные зависимости отсутствуют.

Таблица СменаЦенНаИзделия:

1НФ: соответствует. Все значения атомарны, отсутствуют повторяющиеся атрибуты. Все записи идентифицируются первичным ключом Код.

2НФ: соответствует. Атрибут Цена функционально полно зависит от составного ключа (КодИзделия, ДатаВремя).

3НФ: соответствует. Транзитивные зависимости отсутствуют.

Таблица СоставИзделия:

1НФ: соответствует. Все поля содержат атомарные значения, отсутствуют повторяющиеся атрибуты. Все записи идентифицируются первичным ключом Код.

2НФ: соответствует. Таблица состоит только из ключевых атрибутов КодИзделия и КодИнгредиента, неключевые атрибуты отсутствуют.

3НФ: соответствует. Транзитивные зависимости отсутствуют.

Таблица СоставЗаказа:

1НФ: соответствует. Таблица содержит только атомарные значения, отсутствуют повторяющиеся атрибуты. Все записи идентифицируются первичным ключом Код.

2НФ: соответствует. Атрибут Количество функционально полно зависит от составного ключа (НомерЗаказа, КодИзделия).

3НФ: соответствует. Транзитивные зависимости отсутствуют.

2.7 Построение схемы данных.

Физическую модель базы данных представлена на рисунке 7 и модель из PowerDesigner на рисунке 8.



Рисунок 7 – Физическая модель базы данных

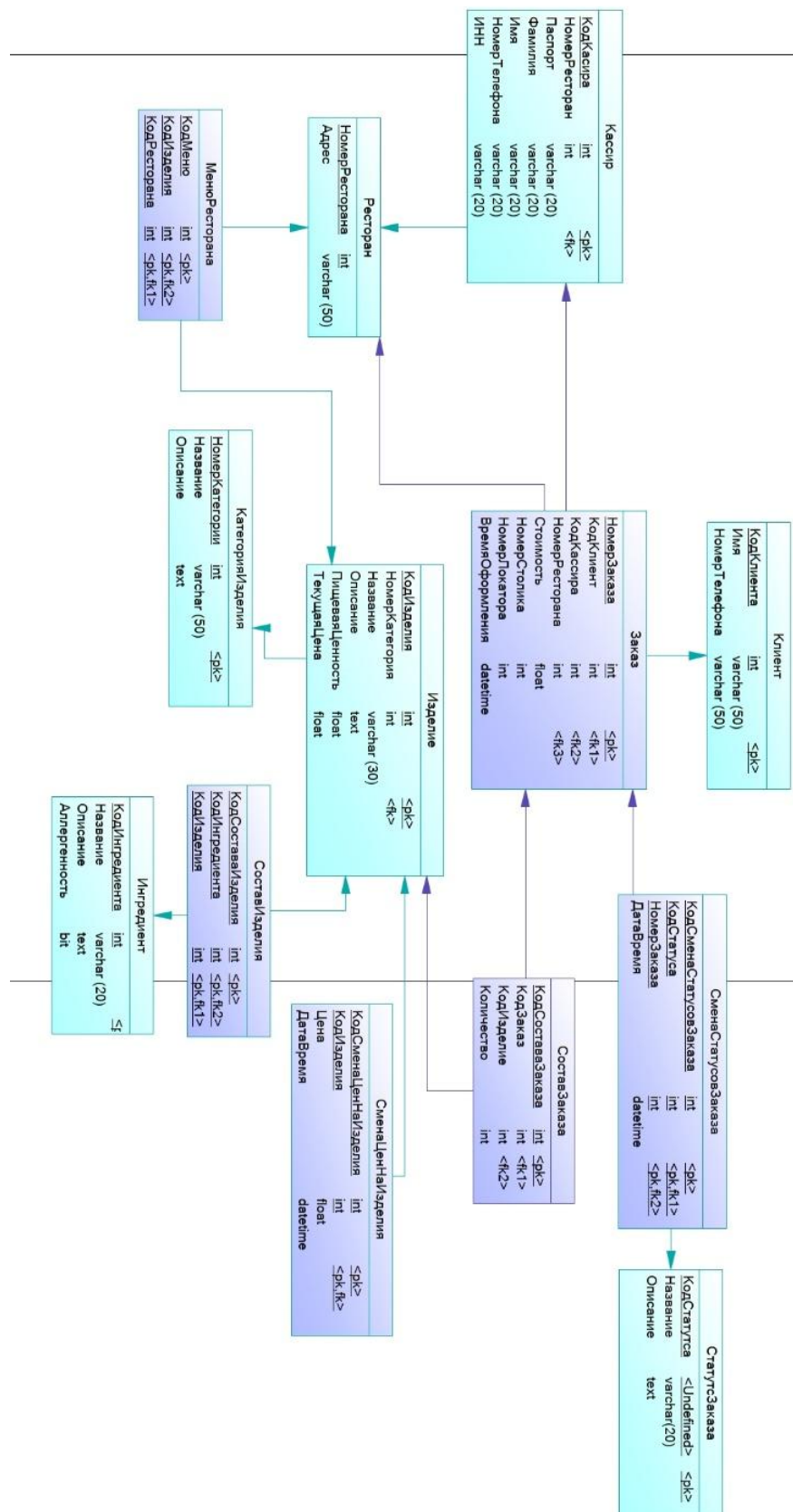


Рисунок 8 – Физическая модель базы данных из PowerDesigner

2.8 Задание частных ограничений целостности данных.

При проектировании БД необходимо учитывать все факторы, влияющие на целостность данных. Далее будут перечислены скрипты создания БД с ограничениями первичных и внешних ключей, а также скрипты для остальных ограничений:

```
-- Создание таблицы "Клиент"

CREATE TABLE Клиент (
    КодКлиента INT PRIMARY KEY,
    Имя VARCHAR(15) NOT NULL,
    НомерТелефона CHAR(11) NOT NULL
    CHECK (
        НомерТелефона ~ '^[0-9]{11}$'
    )
);

-- Создание таблицы "Ресторан"

CREATE TABLE Ресторан (
    НомерРесторана INT PRIMARY KEY,
    Адрес VARCHAR(50) NOT NULL
);

-- Создание таблицы "Кассир"

CREATE TABLE Кассир (
    КодКассира INT PRIMARY KEY,
    Паспорт CHAR(10) NOT NULL
    CHECK (
        Паспорт ~ '^[0-9]{10}$'
    ),
    НомерРесторана INT,
    Фамилия VARCHAR(20) NOT NULL,
    Имя VARCHAR(15) NOT NULL,
    НомерТелефона CHAR(11) NOT NULL
    CHECK (
        НомерТелефона ~ '^[0-9]{11}$'
    ),
    ИНН CHAR(12) NOT NULL
    CHECK (
        ИНН ~ '^[0-9]{12}$'
    ),
    FOREIGN KEY (НомерРесторана)
    REFERENCES Ресторан(НомерРесторана)
);

-- Создание таблицы "КатегорияИзделия"

CREATE TABLE КатегорияИзделия (
    НомерКатегории INT PRIMARY KEY,
    Название VARCHAR(15) NOT NULL UNIQUE,
    Описание VARCHAR(50)
);

-- Создание таблицы "Изделие"

CREATE TABLE Изделие (
    КодИзделия INT PRIMARY KEY,
    НомерКатегории INT NOT NULL,
    Название VARCHAR(30) NOT NULL UNIQUE,
    Описание VARCHAR(200),
```

```

        ПищеваяЦенность INT NOT NULL,
        ТекущаяСтоимость FLOAT,
        FOREIGN KEY (НомерКатегории)
            REFERENCES КатегорияИзделия(НомерКатегории)
    );

-- Создание таблицы "Ингредиент"
CREATE TABLE Ингредиент (
    КодИнгредиента INT PRIMARY KEY,
    Название VARCHAR(15) NOT NULL UNIQUE,
    Описание VARCHAR(50),
    Аллергенность BOOLEAN NOT NULL DEFAULT FALSE
);

-- Создание таблицы "СтатусЗаказа"
CREATE TABLE СтатусЗаказа (
    КодСтатуса INT PRIMARY KEY,
    Название VARCHAR(15) NOT NULL UNIQUE,
    Описание VARCHAR(50)
);

-- Создание таблицы "Заказ"
CREATE TABLE Заказ (
    НомерЗаказа INT PRIMARY KEY,
    КодКлиента INT NOT NULL,
    КодКассира INT NOT NULL,
    НомерРесторана INT NOT NULL,
    Стоимость FLOAT NOT NULL,
    НомерСтолика INT,
    НомерЛокатора INT,
    ВремяОформления TIMESTAMP NOT NULL,
    FOREIGN KEY (КодКлиента)
        REFERENCES Клиент(КодКлиента),
    FOREIGN KEY (КодКассира)
        REFERENCES Кассир(КодКассира),
    FOREIGN KEY (НомерРесторана)
        REFERENCES Ресторан(НомерРесторана)
);

-- Создание таблицы "СменаСтатусовЗаказа"
CREATE TABLE СменаСтатусовЗаказа (
    Код INT PRIMARY KEY,
    НомерЗаказа INT NOT NULL,
    ДатаВремя TIMESTAMP NOT NULL,
    КодСтатуса INT NOT NULL,
    FOREIGN KEY (НомерЗаказа)
        REFERENCES Заказ(НомерЗаказа),
    FOREIGN KEY (КодСтатуса)
        REFERENCES СтатусЗаказа(КодСтатуса)
);

-- Создание таблицы "МенюРесторана"
CREATE TABLE МенюРесторана (
    Код INT PRIMARY KEY,
    НомерРесторана INT NOT NULL,
    КодИзделия INT NOT NULL,
    FOREIGN KEY (НомерРесторана)
        REFERENCES Ресторан(НомерРесторана),
    FOREIGN KEY (КодИзделия)
        REFERENCES Изделие(КодИзделия)
);

-- Создание таблицы "СменаЦенНаИзделие"
CREATE TABLE СменаЦенНаИзделие (
    Код INT PRIMARY KEY,
    КодИзделия INT NOT NULL,

```

```

        ДатаВремя TIMESTAMP NOT NULL,
        Цена FLOAT NOT NULL,
        FOREIGN KEY (КодИзделия)
            REFERENCES Изделие(КодИзделия)
    );

-- Создание таблицы "СоставИзделия"
CREATE TABLE СоставИзделия (
    Код INT PRIMARY KEY,
    КодИнгредиента INT NOT NULL,
    КодИзделия INT NOT NULL,
    FOREIGN KEY (КодИнгредиента)
        REFERENCES Ингредиент(КодИнгредиента),
    FOREIGN KEY (КодИзделия)
        REFERENCES Изделие(КодИзделия)
);

-- Создание таблицы "СоставЗаказа"
CREATE TABLE СоставЗаказа (
    Код INT PRIMARY KEY,
    НомерЗаказа INT NOT NULL,
    КодИзделия INT NOT NULL,
    Количество INT NOT NULL DEFAULT 1,
    FOREIGN KEY (НомерЗаказа)
        REFERENCES Заказ(НомерЗаказа),
    FOREIGN KEY (КодИзделия)
        REFERENCES Изделие(КодИзделия)
);

-- Дополнительные ограничения
-- Ограничение: пищевая ценность изделия больше нуля
ALTER TABLE Изделие
ADD CONSTRAINT chk_пищевая_ценность
CHECK (ПищеваяЦенность >= 0);

-- Ограничение: текущая стоимость больше нуля
ALTER TABLE Изделие
ADD CONSTRAINT chk_текущая_стоимость_положительная
CHECK (ТекущаяСтоимость > 0);

-- Ограничение: время оформления заказа не может быть в будущем
ALTER TABLE Заказ
ADD CONSTRAINT chk_время_оформления
CHECK (ВремяОформления <= CURRENT_TIMESTAMP);

-- Ограничение: если есть номер столика – он должен быть больше нуля
ALTER TABLE Заказ
ADD CONSTRAINT chk_стол_положительный
CHECK (НомерСтолика IS NULL OR НомерСтолика > 0);

-- Ограничение: если есть номер локатора – он должен быть больше нуля
ALTER TABLE Заказ
ADD CONSTRAINT chk_локатор_положительный
CHECK (НомерЛокатора IS NULL OR НомерЛокатора > 0);

-- Ограничение: стоимость заказа больше нуля
ALTER TABLE Заказ
ADD CONSTRAINT chk_стоимость_положительная
CHECK (Стоимость > 0);

-- Ограничение: назначение статуса не может быть в будущем
ALTER TABLE СменаСтатусовЗаказа
ADD CONSTRAINT chk_дата_время_не_в_будущем
CHECK (ДатаВремя <= CURRENT_TIMESTAMP);

-- Ограничение: смена цены не может быть в будущем

```



```

ALTER TABLE СменаЦенНаИзделие
ADD CONSTRAINT chk_дата_время_не_в_будущем
CHECK (ДатаВремя <= CURRENT_TIMESTAMP);

-- Ограничение: цена изделия положительна
ALTER TABLE СменаЦенНаИзделие
ADD CONSTRAINT chk_цена_положительная
CHECK (Цена > 0);

-- Ограничение: количество изделий добавляющихся в заказ положительно
ALTER TABLE СоставЗаказа
ADD CONSTRAINT chk_количество_положительное
CHECK (Количество > 0);

```

2.9 Разработка триггеров и хранимых процедур

В ходе разработки пользовательского функционала, для взаимодействия с базой данных через клиент-серверное приложения, были разработаны следующие хранимые процедуры.

1. Получение списка изделий по их категории

```

CREATE OR REPLACE FUNCTION get_изделия_по_категории(
    p_номер_категории INT
)
RETURNS SETOF Изделие
LANGUAGE sql STABLE AS $$
    SELECT *
    FROM Изделие
    WHERE НомерКатегории = p_номер_категории;
$$;

```

2. Получение данных изделия по его ID

```

CREATE OR REPLACE FUNCTION get_изделие_по_id(
    p_код_изделия INT
)
RETURNS SETOF Изделие
LANGUAGE sql STABLE AS $$
    SELECT *
    FROM Изделие
    WHERE КодИзделия = p_код_изделия;
$$;

```

3. Незавершённые и неотменённые заказы заданного клиента

```

CREATE OR REPLACE FUNCTION get_активные_заказы_клиента(
    p_код_клиента INT
)
RETURNS SETOF Заказ
LANGUAGE sql STABLE AS $$
    WITH последний_статус AS (
        SELECT DISTINCT ON (НомерЗаказа)
            НомерЗаказа,
            КодСтатуса
        FROM СменаСтатусовЗаказа
        ORDER BY НомерЗаказа, ДатаВремя DESC
    )
    SELECT з.*
    FROM Заказ з
    LEFT JOIN последний_статус пс ON з.НомерЗаказа = пс.НомерЗаказа
    LEFT JOIN СтатусЗаказа сс ON пс.КодСтатуса = сс.КодСтатуса

```

```

WHERE з.КодКлиента = р_код_клиента
AND (сс.Название IS NULL
OR сс.Название NOT IN ('Завершён', 'Отменён'));
$$;

```

4. Получение состава заказа по его ID

```

CREATE OR REPLACE FUNCTION get_состав_заказа(
  р_номер_заказа INT
)
RETURNS TABLE (
  Код INT,
  НомерЗаказа INT,
  КодИзделия INT,
  Количество INT,
  НазваниеИзделия VARCHAR(30),
  ТекущаяСтоимость FLOAT
)
LANGUAGE sql STABLE AS $$
  SELECT сз.Код,
         сз.НомерЗаказа,
         сз.КодИзделия,
         сз.Количество,
         и.Название AS НазваниеИзделия,
         и.ТекущаяСтоимость
  FROM СоставЗаказа сз
  JOIN Изделие и ON сз.КодИзделия = и.КодИзделия
  WHERE сз.НомерЗаказа = р_номер_заказа;
$$;

```

5. Завершённые или отменённые заказы заданного клиента

```

CREATE OR REPLACE FUNCTION get_завершённые_заказы_клиента(
  р_код_клиента INT
)
RETURNS SETOF Заказ
LANGUAGE sql STABLE AS $$
  WITH последний_статус AS (
    SELECT DISTINCT ON (НомерЗаказа)
      НомерЗаказа,
      КодСтатуса
    FROM СменаСтатусовЗаказа
    ORDER BY НомерЗаказа, ДатаВремя DESC
  )
  SELECT з.*
  FROM Заказ з
  JOIN последний_статус пс ON з.НомерЗаказа = пс.НомерЗаказа
  JOIN СтатусЗаказа сс ON пс.КодСтатуса = сс.КодСтатуса
  WHERE з.КодКлиента = р_код_клиента
  AND сс.Название IN ('Завершён', 'Отменён');
$$;

```

6. История смены статусов заказа по его ID

```

CREATE OR REPLACE FUNCTION get_история_статусов_заказа(
  р_номер_заказа INT
)
RETURNS TABLE (
  Код INT,
  НомерЗаказа INT,
  ДатаВремя TIMESTAMPTZ,
  КодСтатуса INT,
  НазваниеСтатуса VARCHAR(15),
  ОписаниеСтатуса VARCHAR(50)
)
LANGUAGE sql STABLE AS $$
  SELECT сз.Код,
         сз.НомерЗаказа,

```

```

        сз.ДатаВремя,
        сз.КодСтатуса,
        сс.Название AS НазваниеСтатуса,
        сс.Описание AS ОписаниеСтатуса
FROM   СменаСтатусовЗаказа сз
JOIN   СтатусЗаказа      сс ON сз.КодСтатуса = сс.КодСтатуса
WHERE  сз.НомерЗаказа = р_номер_заказа
ORDER BY сз.ДатаВремя;
$$;

```

7. Добавление нового статуса заказа по его ID

```

CREATE OR REPLACE PROCEDURE add_статус_заказа(
    р_номер_заказа INT,
    р_код_статуса INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_код INT;
BEGIN
    SELECT COALESCE(MAX(Код), 0) + 1
    INTO   v_код
    FROM   СменаСтатусовЗаказа;

    INSERT INTO СменаСтатусовЗаказа(Код, НомерЗаказа, ДатаВремя, КодСтатуса)
    VALUES (v_код, р_номер_заказа, CURRENT_TIMESTAMP, р_код_статуса);
END;
$$;

```

8. Добавление изделия и количества в состав заказа по его ID

```

CREATE OR REPLACE PROCEDURE add_изделие_в_заказ(
    р_номер_заказа INT,
    р_код_изделия INT,
    р_количество INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_код INT;
BEGIN
    SELECT COALESCE(MAX(Код), 0) + 1
    INTO   v_код
    FROM   СоставЗаказа;

    INSERT INTO СоставЗаказа(Код, НомерЗаказа, КодИзделия, Количество)
    VALUES (v_код, р_номер_заказа, р_код_изделия, р_количество);
END;
$$;

```

9. Создание нового заказа с номером столика, без локатора

```

CREATE OR REPLACE FUNCTION create_заказ_co_столиком(
    р_код_клиента INT,
    р_код_кассира INT,
    р_номер_ресторана INT,
    р_стоимость FLOAT,
    р_номер_столика INT
)
RETURNS INT
LANGUAGE plpgsql AS $$
DECLARE
    v_номер_заказа INT;
BEGIN
    SELECT COALESCE(MAX(НомерЗаказа), 0) + 1
    INTO   v_номер_заказа
    FROM   Заказ;

    INSERT INTO Заказ(

```

```

        НомерЗаказа, КодКлиента, КодКассира,
        НомерРесторана, Стоимость,
        НомерСтолика, НомерЛокатора, ВремяОформления
    )
VALUES (
    v_номер_заказа, p_код_клиента, p_код_кассира,
    p_номер_ресторана, p_стоимость,
    p_номер_столика, NULL, CURRENT_TIMESTAMP
);

RETURN v_номер_заказа;
END;
$$;

```

10. Создание нового заказа с номером локатора, без столика

```

CREATE OR REPLACE FUNCTION create_заказ_с_локатором(
    p_код_клиента INT,
    p_код_кассира INT,
    p_номер_ресторана INT,
    p_стоимость FLOAT,
    p_номер_локатора INT
)
RETURNS INT
LANGUAGE plpgsql AS $$
DECLARE
    v_номер_заказа INT;
BEGIN
    SELECT COALESCE(MAX(НомерЗаказа), 0) + 1
    INTO v_номер_заказа
    FROM Заказ;

    INSERT INTO Заказ(
        НомерЗаказа, КодКлиента, КодКассира,
        НомерРесторана, Стоимость,
        НомерСтолика, НомерЛокатора, ВремяОформления
    )
VALUES (
        v_номер_заказа, p_код_клиента, p_код_кассира,
        p_номер_ресторана, p_стоимость,
        NULL, p_номер_локатора, CURRENT_TIMESTAMP
    );

    RETURN v_номер_заказа;
END;
$$;

```

11. Создание нового заказа без столика и без локатора

```

CREATE OR REPLACE FUNCTION create_заказ(
    p_код_клиента INT,
    p_код_кассира INT,
    p_номер_ресторана INT,
    p_стоимость FLOAT
)
RETURNS INT
LANGUAGE plpgsql AS $$
DECLARE
    v_номер_заказа INT;
BEGIN
    SELECT COALESCE(MAX(НомерЗаказа), 0) + 1
    INTO v_номер_заказа
    FROM Заказ;

    INSERT INTO Заказ(
        НомерЗаказа, КодКлиента, КодКассира,
        НомерРесторана, Стоимость,

```

```

        НомерСтолика, НомерЛокатора, ВремяОформления
    )
    VALUES (
        v_номер_заказа, p_код_клиента, p_код_кассира,
        p_номер_ресторана, p_стоимость,
        NULL, NULL, CURRENT_TIMESTAMP
    );

    RETURN v_номер_заказа;
END;
$$;

```

12. Незавершённые и неотменённые заказы заданного кассира

```

CREATE OR REPLACE FUNCTION get_активные_заказы_кассира(
    p_код_кассира INT
)
RETURNS SETOF Заказ
LANGUAGE sql STABLE AS $$
    WITH последний_статус AS (
        SELECT DISTINCT ON (НомерЗаказа)
            НомерЗаказа,
            КодСтатуса
        FROM СменаСтатусовЗаказа
        ORDER BY НомерЗаказа, ДатаВремя DESC
    )
    SELECT з.*
    FROM Заказ з
    LEFT JOIN последний_статус пс ON з.НомерЗаказа = пс.НомерЗаказа
    LEFT JOIN СтатусЗаказа сс ON пс.КодСтатуса = сс.КодСтатуса
    WHERE з.КодКассира = p_код_кассира
    AND (сс.Название IS NULL
        OR сс.Название NOT IN ('Завершён', 'Отменён'));
$$;

```

13. Завершённые или отменённые заказы заданного кассира

```

CREATE OR REPLACE FUNCTION get_завершённые_заказы_кассира(
    p_код_кассира INT
)
RETURNS SETOF Заказ
LANGUAGE sql STABLE AS $$
    WITH последний_статус AS (
        SELECT DISTINCT ON (НомерЗаказа)
            НомерЗаказа,
            КодСтатуса
        FROM СменаСтатусовЗаказа
        ORDER BY НомерЗаказа, ДатаВремя DESC
    )
    SELECT з.*
    FROM Заказ з
    JOIN последний_статус пс ON з.НомерЗаказа = пс.НомерЗаказа
    JOIN СтатусЗаказа сс ON пс.КодСтатуса = сс.КодСтатуса
    WHERE з.КодКассира = p_код_кассира
    AND сс.Название IN ('Завершён', 'Отменён');
$$;

```

14. Получение данных кассира по его ID

```

CREATE OR REPLACE FUNCTION get_кассир_по_id(
    p_код_кассира INT
)
RETURNS SETOF Кассир
LANGUAGE sql STABLE AS $$
    SELECT *
    FROM Кассир
    WHERE КодКассира = p_код_кассира;
$$;

```

15. Получение списка всех кассиров

```
CREATE OR REPLACE FUNCTION get_все_кассиры()
RETURNS SETOF Кассир
LANGUAGE sql STABLE AS $$
    SELECT * FROM Кассир;
$$;
```

16. Получение списка всех категорий

```
CREATE OR REPLACE FUNCTION get_все_категории()
RETURNS SETOF КатегорияИзделия
LANGUAGE sql STABLE AS $$
    SELECT * FROM КатегорияИзделия;
$$;
```

17. Получение списка всех ресторанов

```
CREATE OR REPLACE FUNCTION get_все_рестораны()
RETURNS SETOF Ресторан
LANGUAGE sql STABLE AS $$
    SELECT * FROM Ресторан;
$$;
```

18. Создание нового кассира

```
CREATE OR REPLACE PROCEDURE create_кассир(
    p_паспорт      CHAR(10),
    p_номер_ресторана INT,
    p_фамилия      VARCHAR(20),
    p_имя          VARCHAR(15),
    p_номер_телефона CHAR(11),
    p_инн          CHAR(12)
)
LANGUAGE plpgsql AS $$
DECLARE
    v_код INT;
BEGIN
    SELECT COALESCE(MAX(КодКассира), 0) + 1
    INTO   v_код
    FROM   Кассир;

    INSERT INTO Кассир(
        КодКассира, Паспорт, НомерРесторана,
        Фамилия, Имя, НомерТелефона, ИНН
    )
    VALUES (
        v_код, p_паспорт, p_номер_ресторана,
        p_фамилия, p_имя, p_номер_телефона, p_инн
    );
END;
$$;
```

19. Изменение данных кассира

```
CREATE OR REPLACE PROCEDURE update_кассир(
    p_код_кассира INT,
    p_паспорт      CHAR(10),
    p_номер_ресторана INT,
    p_фамилия      VARCHAR(20),
    p_имя          VARCHAR(15),
    p_номер_телефона CHAR(11),
    p_инн          CHAR(12)
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE Кассир
    SET   Паспорт      = p_паспорт,
```

```

        НомерРесторана = p_номер_ресторана,
        Фамилия        = p_фамилия,
        Имя             = p_имя,
        НомерТелефона   = p_номер_телефона,
        ИНН             = p_инн
    WHERE КодКассира = p_код_кассира;
END;
$$;

```

20. Получение списка всех изделий

```

CREATE OR REPLACE FUNCTION get_все_изделия()
RETURNS SETOF Изделие
LANGUAGE sql STABLE AS $$
    SELECT * FROM Изделие;
$$;

```

21. Создание новой категории

```

CREATE OR REPLACE PROCEDURE create_категория(
    p_название VARCHAR(15),
    p_описание VARCHAR(50)
)
LANGUAGE plpgsql AS $$
DECLARE
    v_номер INT;
BEGIN
    SELECT COALESCE(MAX(НомерКатегории), 0) + 1
    INTO v_номер
    FROM КатегорияИзделия;

    INSERT INTO КатегорияИзделия(НомерКатегории, Название, Описание)
    VALUES (v_номер, p_название, p_описание);
END;
$$;

```

22. Изменение данных категории

```

CREATE OR REPLACE PROCEDURE update_категория(
    p_номер_категории INT,
    p_название VARCHAR(15),
    p_описание VARCHAR(50)
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE КатегорияИзделия
    SET Название = p_название,
        Описание = p_описание
    WHERE НомерКатегории = p_номер_категории;
END;
$$;

```

23. Удаление категории, если она не указана ни у одного изделия

```

CREATE OR REPLACE PROCEDURE delete_категория(
    p_номер_категории INT
)
LANGUAGE plpgsql AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM Изделие WHERE НомерКатегории = p_номер_категории
    ) THEN
        RAISE EXCEPTION
        'Категория % не может быть удалена: на неё ссылаются изделия',
        p_номер_категории;
    END IF;

    DELETE FROM КатегорияИзделия WHERE НомерКатегории = p_номер_категории;
END;

```

\$\$;

24. Получение списка всех ингредиентов

```
CREATE OR REPLACE FUNCTION get_все_ингредиенты()  
RETURNS SETOF Ингредиент  
LANGUAGE sql STABLE AS $$  
    SELECT * FROM Ингредиент;  
$$;
```

25. Удаление ингредиента, если он не указан ни у одного изделия

```
CREATE OR REPLACE PROCEDURE delete_ингредиент(  
    p_код_ингредиента INT  
)  
LANGUAGE plpgsql AS $$  
BEGIN  
    IF EXISTS (  
        SELECT 1 FROM СоставИзделия WHERE КодИнгредиента = p_код_ингредиента  
    ) THEN  
        RAISE EXCEPTION  
            'Ингредиент % не может быть удалён: он входит в состав изделий',  
            p_код_ингредиента;  
    END IF;  
  
    DELETE FROM Ингредиент WHERE КодИнгредиента = p_код_ингредиента;  
END;  
$$;
```

26. Создание нового ингредиента

```
CREATE OR REPLACE PROCEDURE create_ингредиент(  
    p_название VARCHAR(15),  
    p_описание VARCHAR(50),  
    p_аллергенность BOOLEAN  
)  
LANGUAGE plpgsql AS $$  
DECLARE  
    v_код INT;  
BEGIN  
    SELECT COALESCE(MAX(КодИнгредиента), 0) + 1  
    INTO v_код  
    FROM Ингредиент;  
  
    INSERT INTO Ингредиент(КодИнгредиента, Название, Описание, Аллергенность)  
    VALUES (v_код, p_название, p_описание, p_аллергенность);  
END;  
$$;
```

27. Изменение данных ингредиента

```
CREATE OR REPLACE PROCEDURE update_ингредиент(  
    p_код_ингредиента INT,  
    p_название VARCHAR(15),  
    p_описание VARCHAR(50),  
    p_аллергенность BOOLEAN  
)  
LANGUAGE plpgsql AS $$  
BEGIN  
    UPDATE Ингредиент  
    SET Название = p_название,  
        Описание = p_описание,  
        Аллергенность = p_аллергенность  
    WHERE КодИнгредиента = p_код_ингредиента;  
END;  
$$;
```

28. Получение состава изделия


```

CREATE OR REPLACE FUNCTION get_состав_изделия(
  p_код_изделия INT
)
RETURNS TABLE (
  Код INT,
  КодИнгредиента INT,
  КодИзделия INT,
  НазваниеИнгредиента VARCHAR(15),
  ОписаниеИнгредиента VARCHAR(50),
  Аллергенность BOOLEAN
)
LANGUAGE sql STABLE AS $$
  SELECT си.Код,
         си.КодИнгредиента,
         си.КодИзделия,
         и.Название AS НазваниеИнгредиента,
         и.Описание AS ОписаниеИнгредиента,
         и.Аллергенность
  FROM СоставИзделия си
  JOIN Ингредиент и ON си.КодИнгредиента = и.КодИнгредиента
  WHERE си.КодИзделия = p_код_изделия;
$$;

```

29. Получение меню изделия (рестораны, в которых оно есть)

```

CREATE OR REPLACE FUNCTION get_меню_изделия(
  p_код_изделия INT
)
RETURNS TABLE (
  Код INT,
  НомерРесторана INT,
  КодИзделия INT,
  АдресРесторана VARCHAR(50)
)
LANGUAGE sql STABLE AS $$
  SELECT мр.Код,
         мр.НомерРесторана,
         мр.КодИзделия,
         р.Адрес AS АдресРесторана
  FROM МенюРесторана мр
  JOIN Ресторан р ON мр.НомерРесторана = р.НомерРесторана
  WHERE мр.КодИзделия = p_код_изделия;
$$;

```

30. Создание нового изделия

```

CREATE OR REPLACE PROCEDURE create_изделие(
  p_номер_категории INT,
  p_название VARCHAR(30),
  p_описание VARCHAR(200),
  p_пищевая_ценность INT,
  p_текущая_стоимость FLOAT
)
LANGUAGE plpgsql AS $$
DECLARE
  v_код INT;
BEGIN
  SELECT COALESCE(MAX(КодИзделия), 0) + 1
  INTO v_код
  FROM Изделие;

  INSERT INTO Изделие(
    КодИзделия, НомерКатегории, Название,
    Описание, ПищеваяЦенность, ТекущаяСтоимость
  )
  VALUES (
    v_код, p_номер_категории, p_название,

```

```

        р_описание, р_пищевая_ценность, р_текущая_стоимость
    );
END;
$$;

```

31. Изменение данных изделия

```

CREATE OR REPLACE PROCEDURE update_изделие(
    р_код_изделия    INT,
    р_номер_категории INT,
    р_название       VARCHAR(30),
    р_описание       VARCHAR(200),
    р_пищевая_ценность INT,
    р_текущая_стоимость FLOAT
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE Изделие
    SET  НомерКатегории = р_номер_категории,
        Название       = р_название,
        Описание       = р_описание,
        ПищеваяЦенность = р_пищевая_ценность,
        ТекущаяСтоимость = р_текущая_стоимость
    WHERE КодИзделия = р_код_изделия;
END;
$$;

```

32. Добавление нового ингредиента в состав изделия

```

CREATE OR REPLACE PROCEDURE add_ингредиент_в_изделие(
    р_код_изделия    INT,
    р_код_ингредиента INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_код INT;
BEGIN
    SELECT COALESCE(MAX(Код), 0) + 1
    INTO  v_код
    FROM  СоставИзделия;

    INSERT INTO СоставИзделия(Код, КодИнгредиента, КодИзделия)
    VALUES (v_код, р_код_ингредиента, р_код_изделия);
END;
$$;

```

33. Удаление ингредиента из состава изделия

```

CREATE OR REPLACE PROCEDURE delete_ингредиент_из_изделия(
    р_код_изделия    INT,
    р_код_ингредиента INT
)
LANGUAGE plpgsql AS $$
BEGIN
    DELETE FROM СоставИзделия
    WHERE КодИзделия = р_код_изделия
    AND КодИнгредиента = р_код_ингредиента;
END;
$$;

```

34. Удаление изделия из меню ресторана

```

CREATE OR REPLACE PROCEDURE delete_изделие_из_меню(
    р_номер_ресторана INT,
    р_код_изделия    INT
)
LANGUAGE plpgsql AS $$
BEGIN
    DELETE FROM МенюРесторана

```

```

WHERE НомерРесторана = p_номер_ресторана
AND КодИзделия = p_код_изделия;
END;
$$;

```

35. Добавление изделия в меню ресторана

```

CREATE OR REPLACE PROCEDURE add_изделие_в_меню(
  p_номер_ресторана INT,
  p_код_изделия INT
)
LANGUAGE plpgsql AS $$
DECLARE
  v_код INT;
BEGIN
  SELECT COALESCE(MAX(Код), 0) + 1
  INTO v_код
  FROM МенюРесторана;

  INSERT INTO МенюРесторана(Код, НомерРесторана, КодИзделия)
  VALUES (v_код, p_номер_ресторана, p_код_изделия);
END;
$$;

```

36. Добавление новой цены изделия + обновление текущей цены

```

CREATE OR REPLACE PROCEDURE add_цена_изделия(
  p_код_изделия INT,
  p_цена FLOAT
)
LANGUAGE plpgsql AS $$
DECLARE
  v_код INT;
BEGIN
  -- Фиксируем смену цены в истории
  SELECT COALESCE(MAX(Код), 0) + 1
  INTO v_код
  FROM СменаЦенНаИзделие;

  INSERT INTO СменаЦенНаИзделие(Код, КодИзделия, ДатаВремя, Цена)
  VALUES (v_код, p_код_изделия, CURRENT_TIMESTAMP, p_цена);

  -- Обновляем текущую стоимость в карточке изделия
  UPDATE Изделие
  SET ТекущаяСтоимость = p_цена
  WHERE КодИзделия = p_код_изделия;
END;
$$;

```

3 Разработка клиентской части информационной системы

3.1 Организация взаимодействия клиентской программы с БД

Взаимодействие клиента с БД осуществляется посредством использования шаблона JDBC и объектов DAO (Data Access Object). Подключение к БД реализовано через Spring DataSource. В этом случае для получения информации о jdbc драйвере используя spring подход, необходимо создать соответствующий data source bean. Созданный data source можно использовать в рамках проекта, предварительно проинжектив его в коде. Для этого в файле properties были прописаны следующие свойства:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/postgres
spring.datasource.username=postgres
spring.datasource.password=123
spring.datasource.driver-class-name=org.postgresql.Driver
```

3.2 Разработка форм

В ходе данной работы были разработаны странички html. Для каждой страницы был написан контроллер, обрабатывающий GET и POST запросы текущего url. На рисунке 9 показана главная страница.

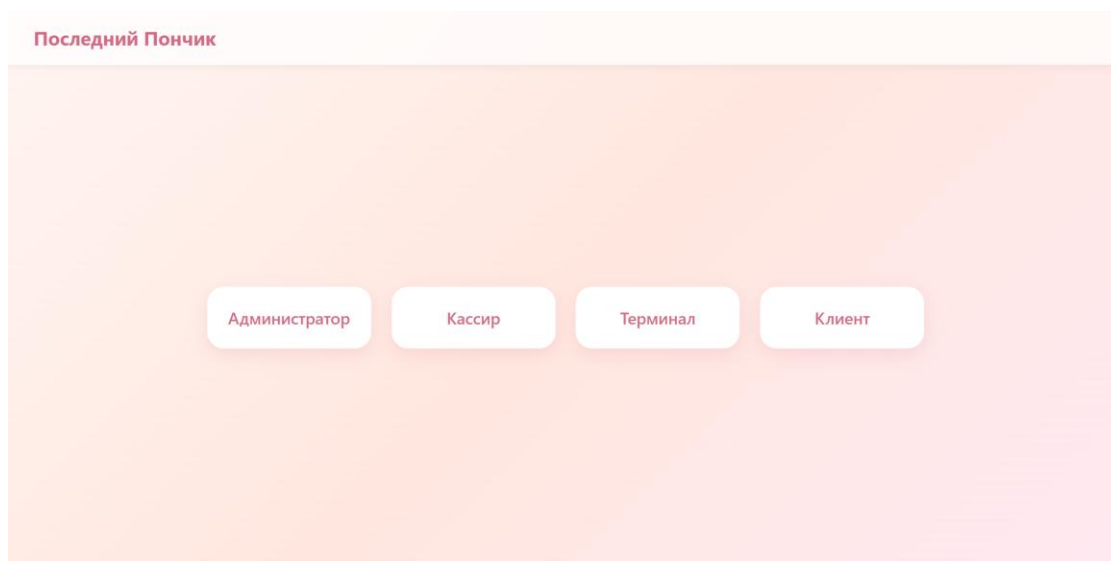


Рисунок 9 – Главная страница

При нажатии кнопок «Администратор», «Кассир» и «Терминал» открывается страница авторизации (рисунок 10), после чего, при успешной авторизации, открывается приложение администратора, кассира и терминала соответственно. При выборе «Клиент» открывается клиентское приложение.

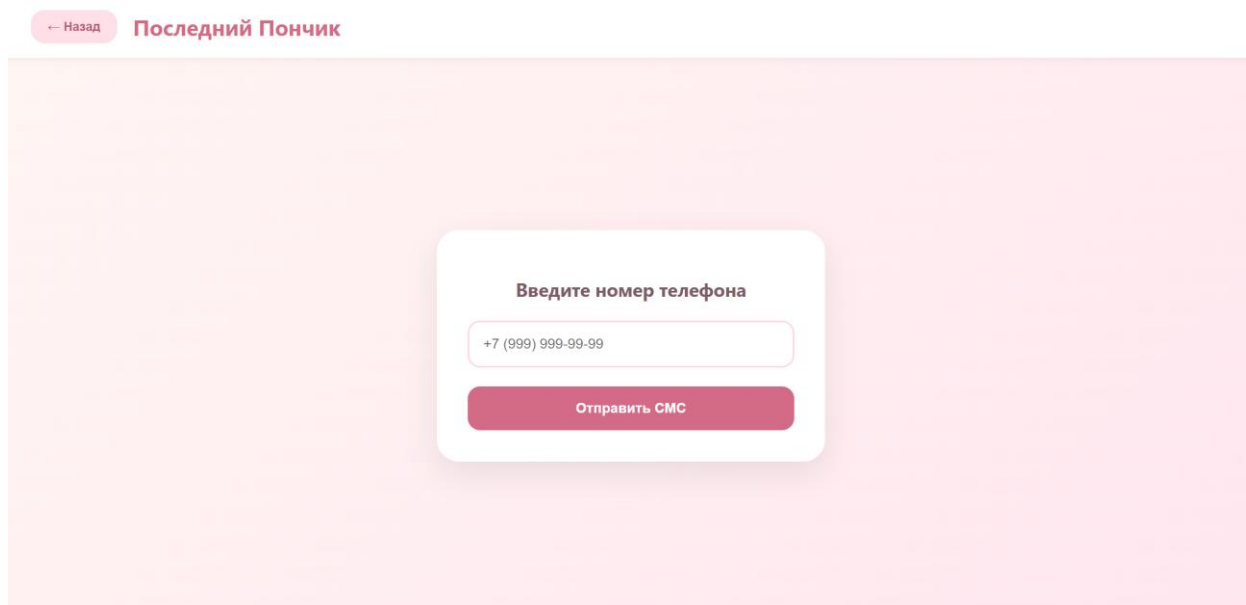
The image shows a mobile application interface for 'Последний Пончик'. At the top, there is a navigation bar with a pink button labeled '← Назад' and the text 'Последний Пончик' in pink. The main background is a light pink gradient. In the center, there is a white rounded rectangle containing the text 'Введите номер телефона' in bold. Below this is a text input field with the placeholder '+7 (999) 999-99-99'. At the bottom of the white box is a pink button with the text 'Отправить СМС'.

Рисунок 10 – Страница авторизации

Приложение клиента

Главная страница с выбором изделий в меню по категориям для неавторизованного клиента приведена на рисунке 11, а для авторизованного – на рисунке 12. При нажатии неавторизованным клиентов кнопки «вход» открывается окно авторизации. Кроме того, на этих страницах у пользователя есть возможность выбрать ресторан, в котором он хочет сделать заказ.

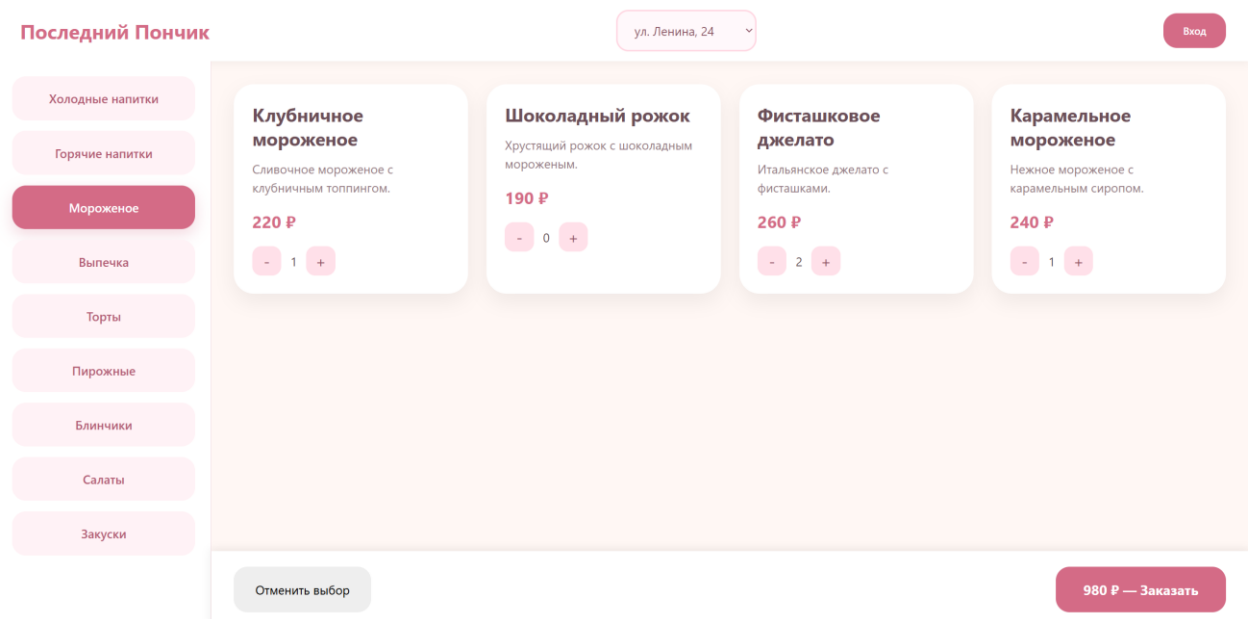


Рисунок 11 – Главная страница неавторизованного клиента

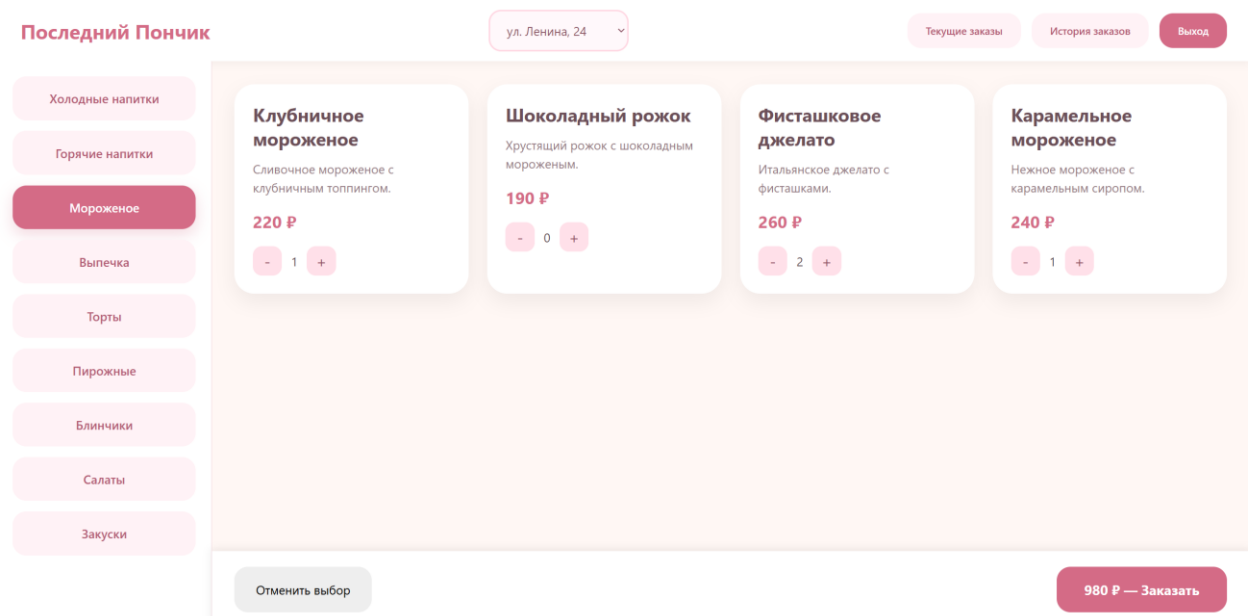


Рисунок 12 – Главная страница авторизованного клиента

При нажатии кнопки «Заказать» открывается страница для просмотра данных заказа – корзина (рисунок 13). У пользователя есть возможность указать номер столика, к которому принесут заказ, если он не хочет забирать его у кассы самостоятельно.

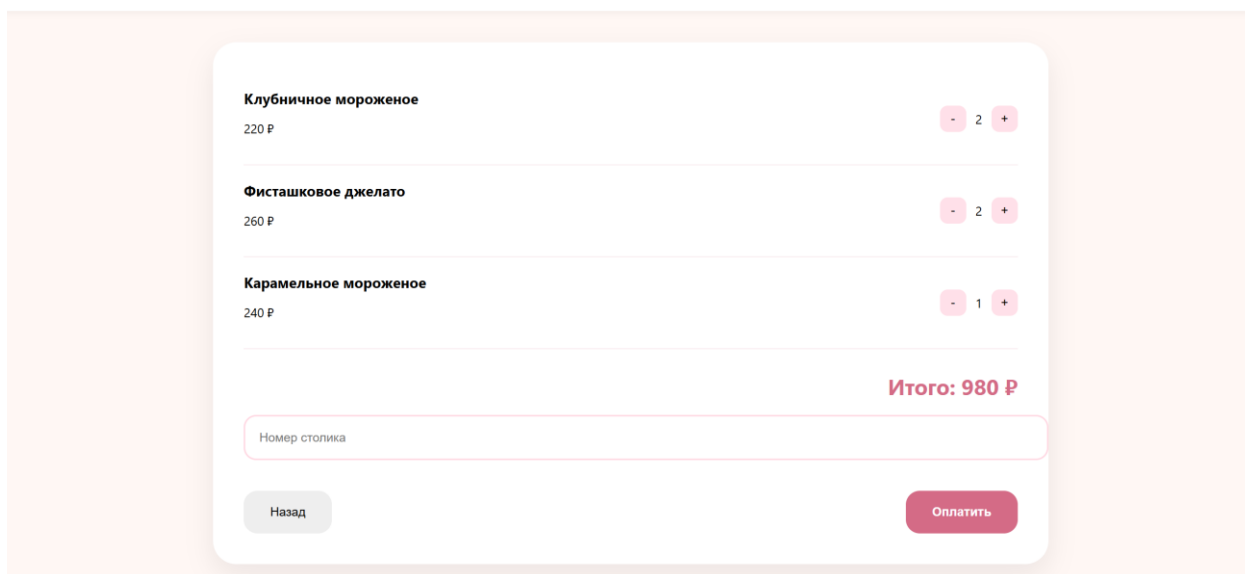


Рисунок 13 – Клиент, страница оформления заказа

При нажатии на главной странице на продукт – открывается карточка изделия с подробной информацией (рисунок 14).

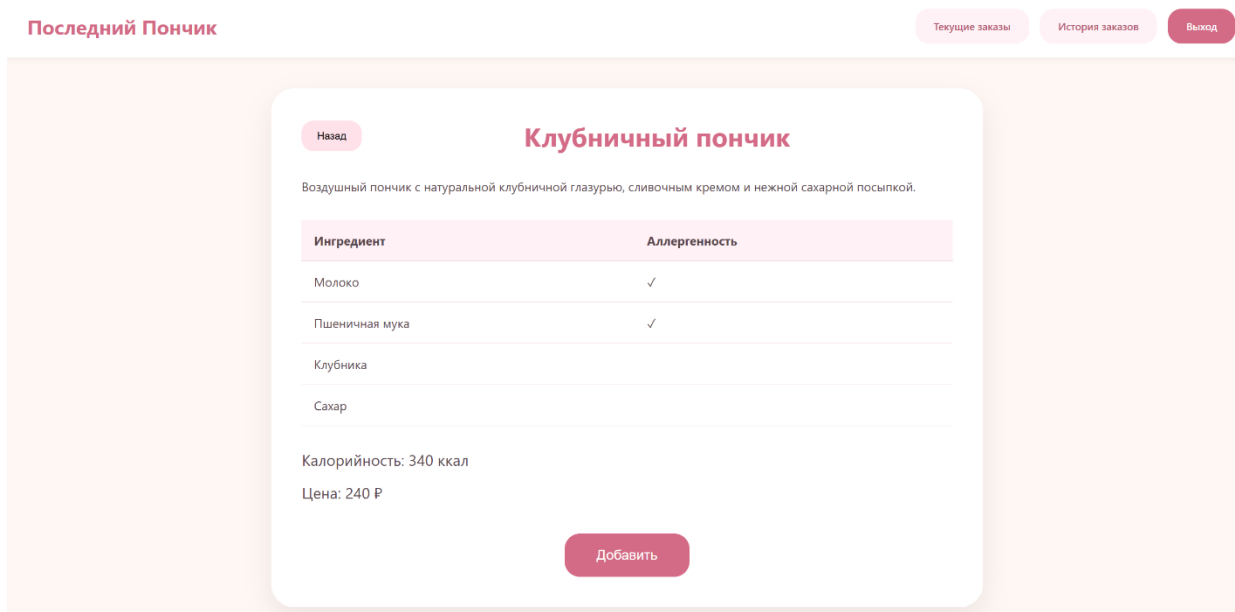


Рисунок 14 – Страница просмотра информации о изделии

При нажатии кнопки «текущие заказы» в шапке страницы открывается список текущих заказов клиента в виде раскрывающихся выпадающих списков (рисунки 15, 16).

[← Назад](#)

Заказ #12	10.05.26 12:30	Столик 12	Готовится
-----------	----------------	-----------	-----------

Заказ #20	10.05.26 12:40	-	Создан
-----------	----------------	---	--------

Рисунок 15 – Текущие заказы клиента

[← Назад](#)

Заказ #12	10.05.26 12:30	Столик 12	Готовится
-----------	----------------	-----------	-----------

Товар	Кол-во	Цена
Фисташковое джелато	2	260 Р
Шоколадный рожок	1	190 Р
		Итого: 710 Р

Заказ #20	10.05.26 12:40	-	Создан
-----------	----------------	---	--------

Товар	Кол-во	Цена
Карамельное мороженое	2	240 Р
Клубничное мороженое	3	220 Р
Блинчики с шоколадом	1	100 Р
		Итого: 1240 Р

Рисунок 16 – Текущие заказы клиента

При нажатии кнопки «история заказов» в шапке страницы открывается список текущих заказов клиента в виде раскрывающихся выпадающих списков (рисунки 17, 18).

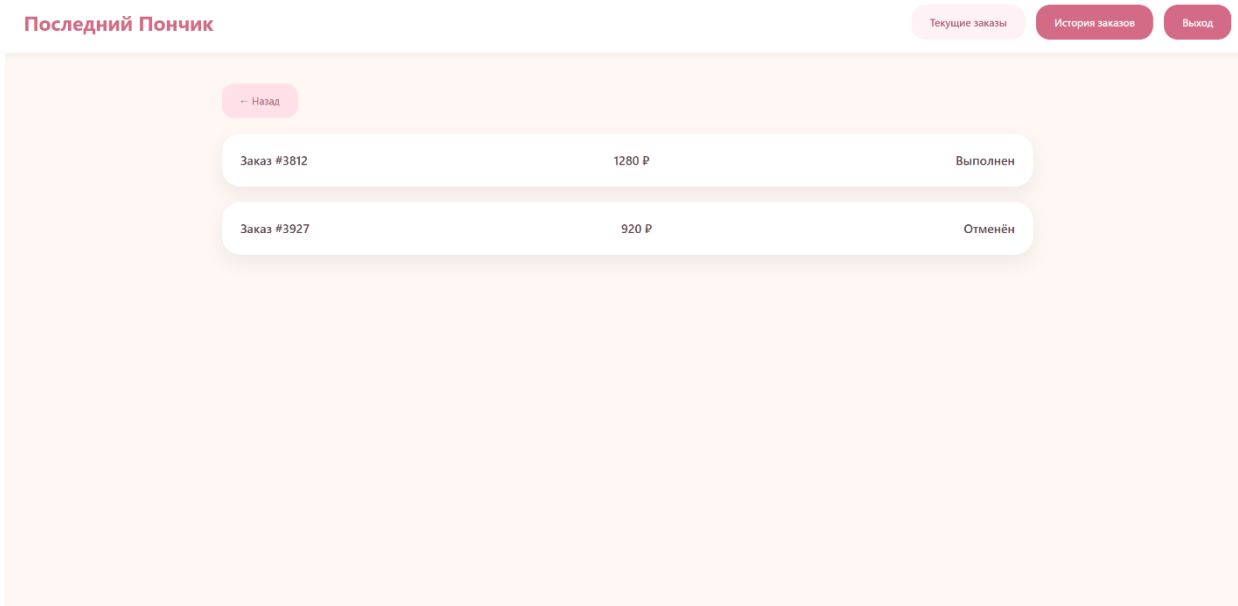


Рисунок 17 – История заказов клиента

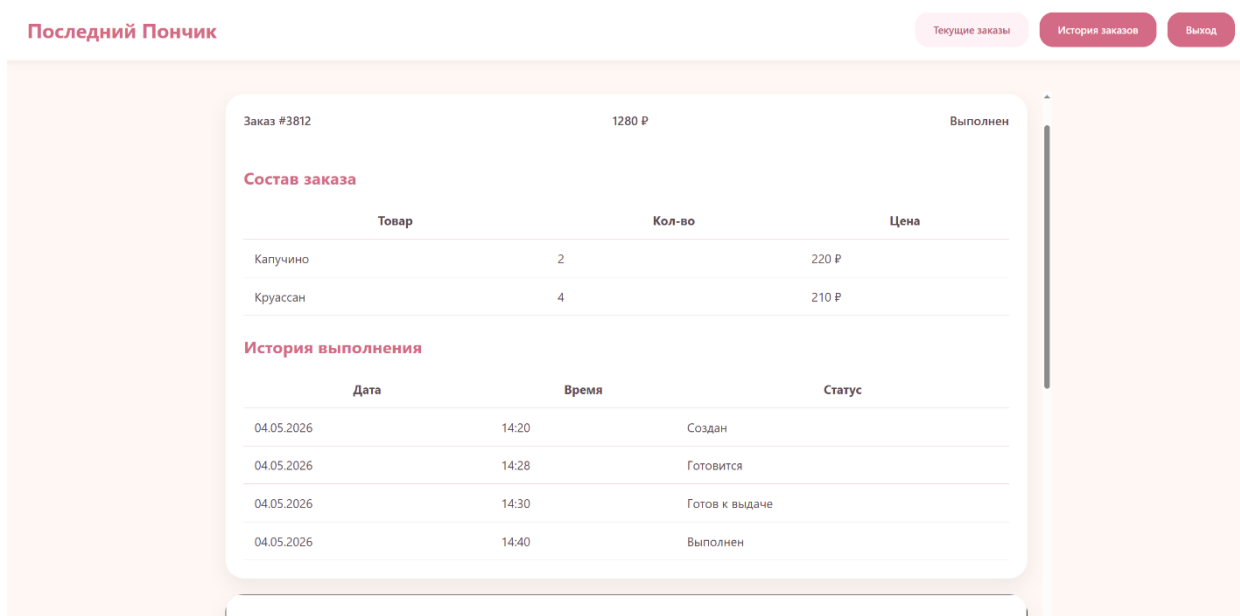


Рисунок 18 – История заказов клиента

Приложение терминала

Приложение терминала во многом похоже на приложение клиента, однако предоставляет менее обширную функциональность. Примеры различных категорий меню изображены на рисунках 19 и 20. Вид раздела для оформления заказа приведен на рисунке 21 (он отличается от клиентского

выбором номера локатора, а не столика). Карточка изделия, открывающаяся при нажатии на изделие в меню, уже была приведена ранее (см. рисунок 14)

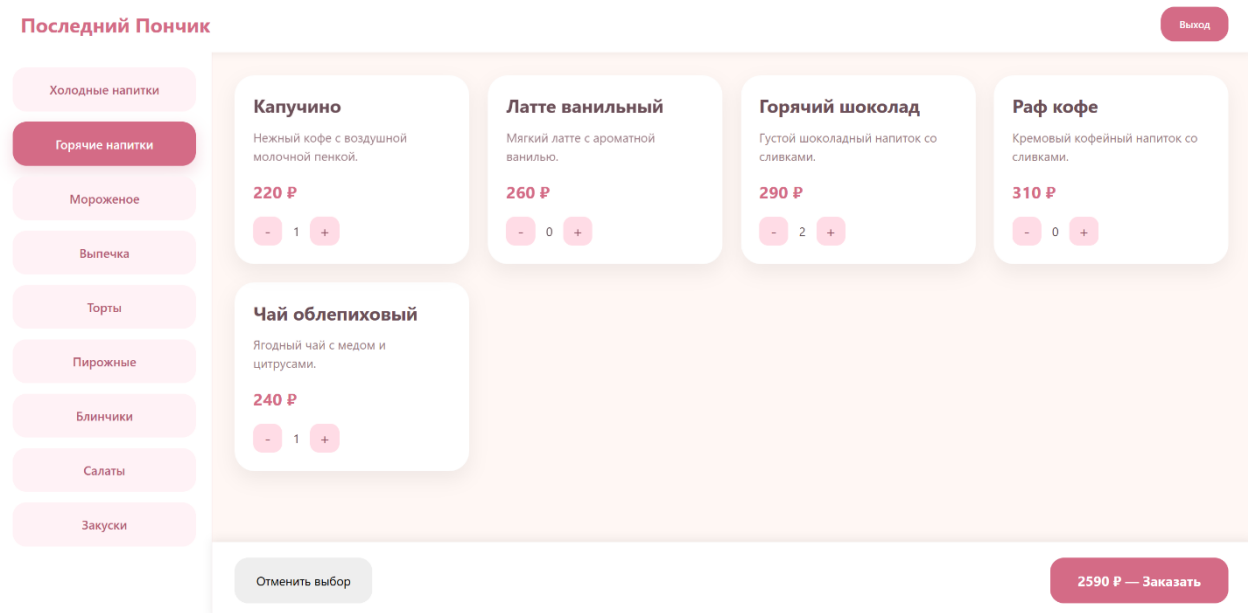


Рисунок 19 – Терминал, главная страница

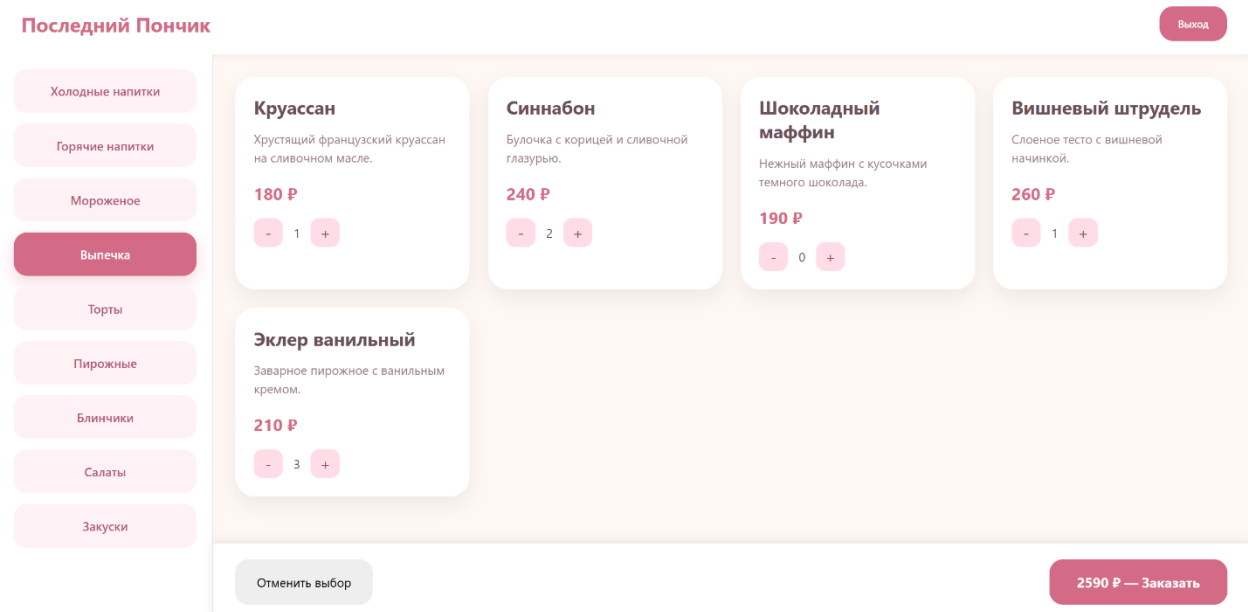


Рисунок 20 – Терминал, главная страница

Клубничный пончик
240 Р

Малиновый торт
540 Р

Итого: 1020 Р

Номер покатора

Назад Оплатить

Рисунок 21 – Терминал, страница оформления заказа

Приложение кассира

На главной странице приложения кассира отображаются текущие заказы, за которыми он следит и статус которых может изменять (рисунок 22). Кроме того, кассиру доступен раздел истории заказов, где он может просматривать выполненные заказы (рисунок 23).

Заказ #25 02.05.26 14:32 Столик 12

Готовится Создан Готовится Готов к выдаче Выполнен Отменен Сохранить

Состав заказа

Товар	Количество	
Капучино	2	220 Р
Круассан	1	180 Р

Итого: 620 Р

История изменений

Дата	Время	Статус
17.05.2026	14:32	Создан
17.05.2026	14:35	Готовится

Заказ #26 02.05.26 14:41 Локатор А-17 Создан Сохранить

Заказ #27 02.05.26 14:58 — Готовится Сохранить

Рисунок 22 – Текущие заказы кассира

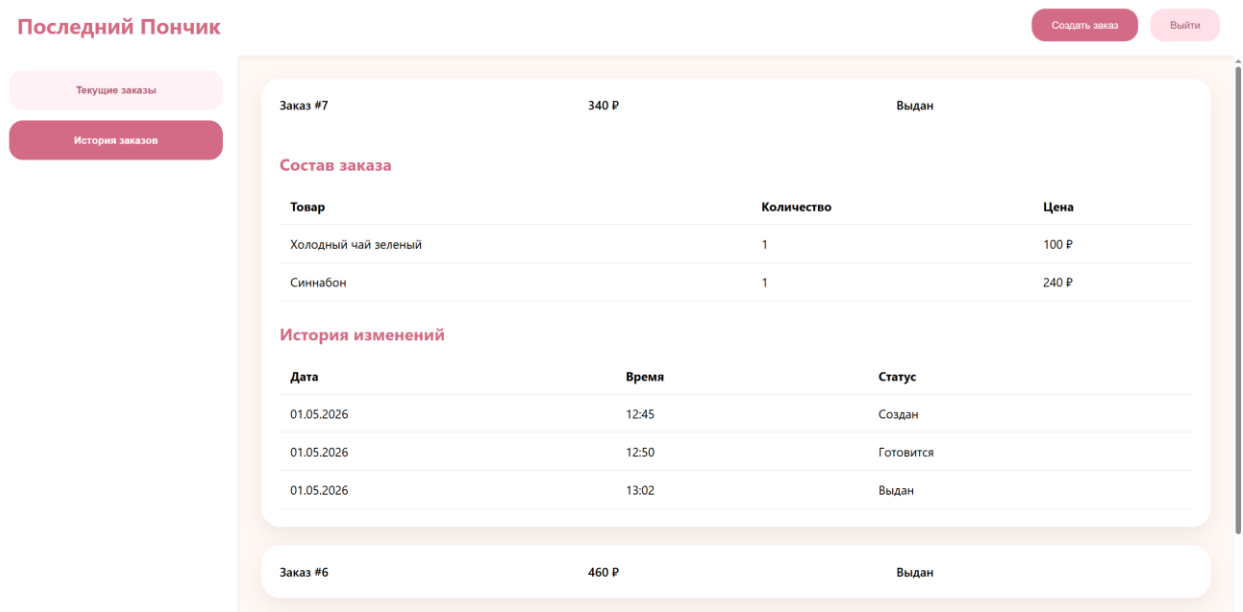


Рисунок 23 – История заказов кассира

Кнопку «создать заказ» кассир будет использовать при необходимости у клиента купить изделия через кассу, а не через приложение или терминал. Интерфейс меню в данном случае аналогичен интерфейсу терминала (рисунок 24). Интерфейс корзины приведен на рисунке 25.

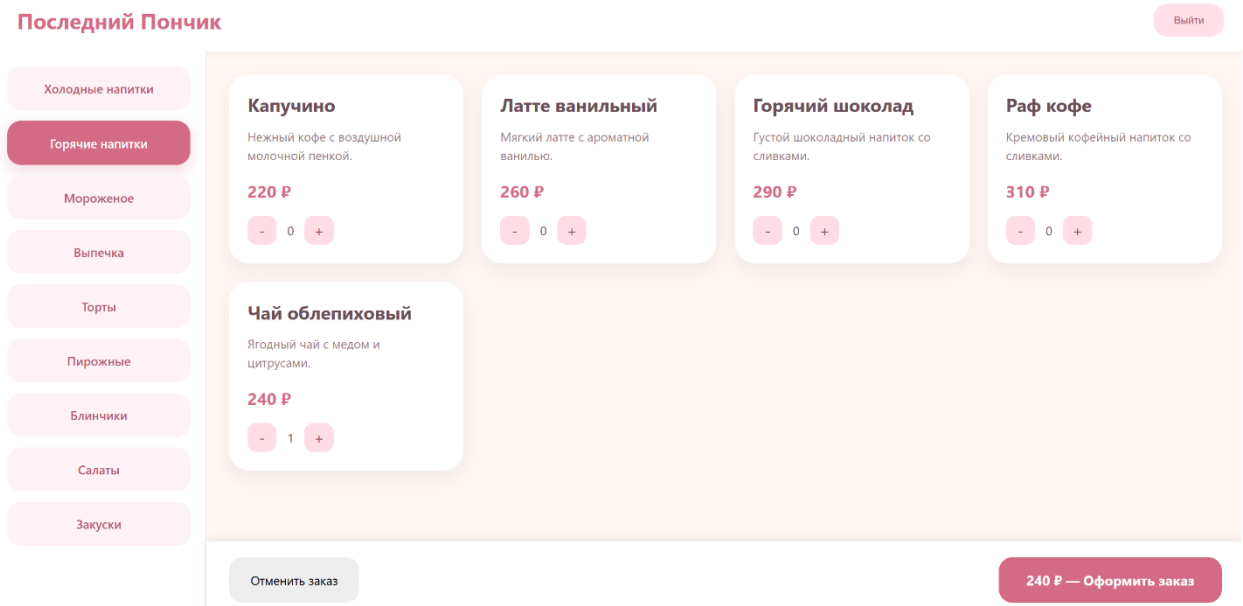


Рисунок 24 – Кассир, страница выбора изделий

Чай облепиховый
240 Р

Итого: 240 Р

Номер телефона

Назад Оформить

Рисунок 25 – Кассир, страница оформления заказа

Приложение администратора

На главной странице администратора отображаются различные разделы, в которых он может проводить изменения. На рисунках 26, 27, 28 показаны раздел кассиров, окно создания нового кассира и окно изменения данных кассира соответственно.

Кассиры

Категории

Ингредиенты

Изделия

Добавить кассира

4510 348921 — Анна Смирнова — ул. Ленина, 24

4512 918245 — Игорь Волков — ул. Горького, 25

4509 441288 — Мария Соколова — ул. Циолковского, 40

Рисунок 26 – Администратор, страница данных кассиров

Последний Пончик

100 % — + Сбросить Выход

Имя

Фамилия

Паспорт

ул. Ленина, 24

Номер телефона

ИНН

Создать Отмена

Рисунок 27 – Администратор, страница создания кассира

Последний Пончик

Выход

Анна

Смирнова

4510348921

ул. Ленина, 24

7 930 458 34 34

012345678912

Сохранить Отмена

Рисунок 28 – Администратор, страница изменения кассира

На рисунках 29, 30, 31 показаны раздел категорий изделий, окно создания новой категории и окно изменения данных категории соответственно.



Рисунок 29 – Администратор, страница данных категорий изделий

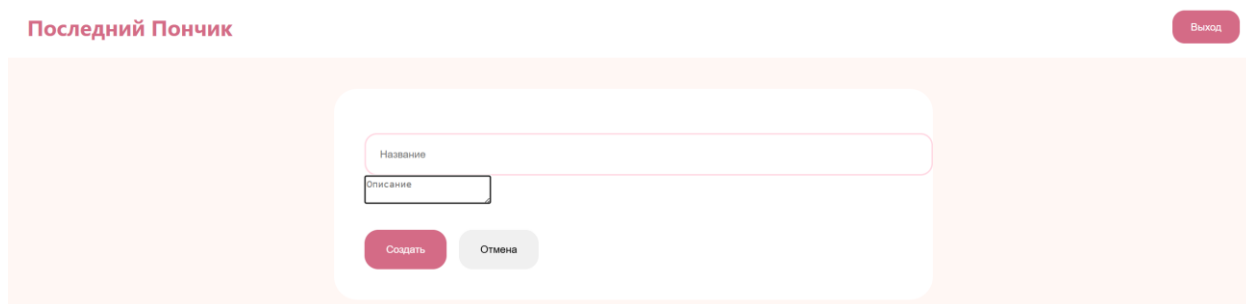


Рисунок 30 – Администратор, страница создания категории изделий

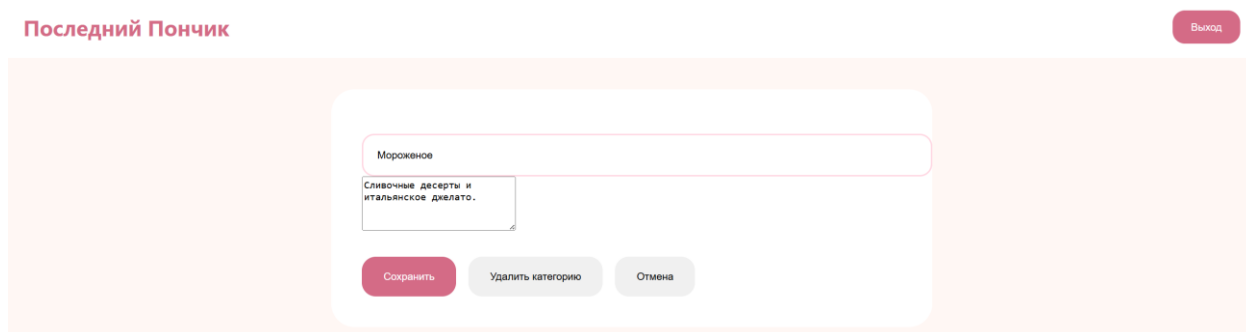


Рисунок 31 – Администратор, страница изменения категории изделий

На рисунках 32, 33, 34 показаны раздел ингредиентов изделий, окно создания нового ингредиента и окно изменения данных ингредиента соответственно.



Рисунок 32 – Администратор, страница данных ингредиентов изделий

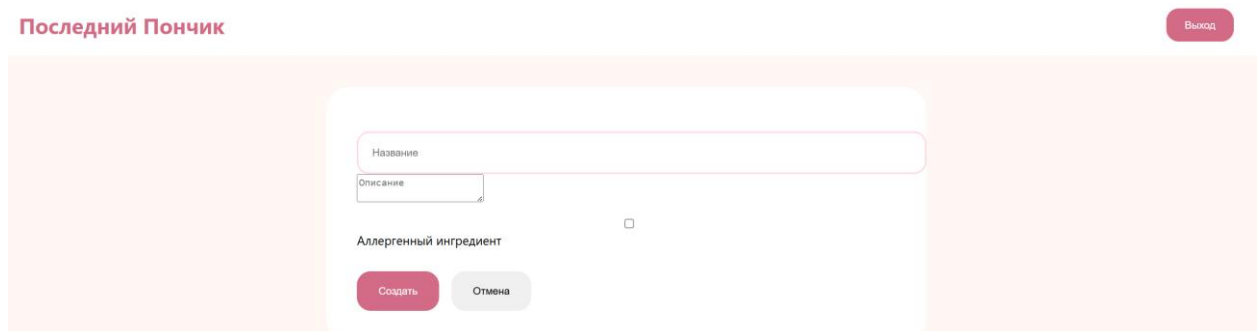


Рисунок 33 – Администратор, страница создания ингредиентов изделий

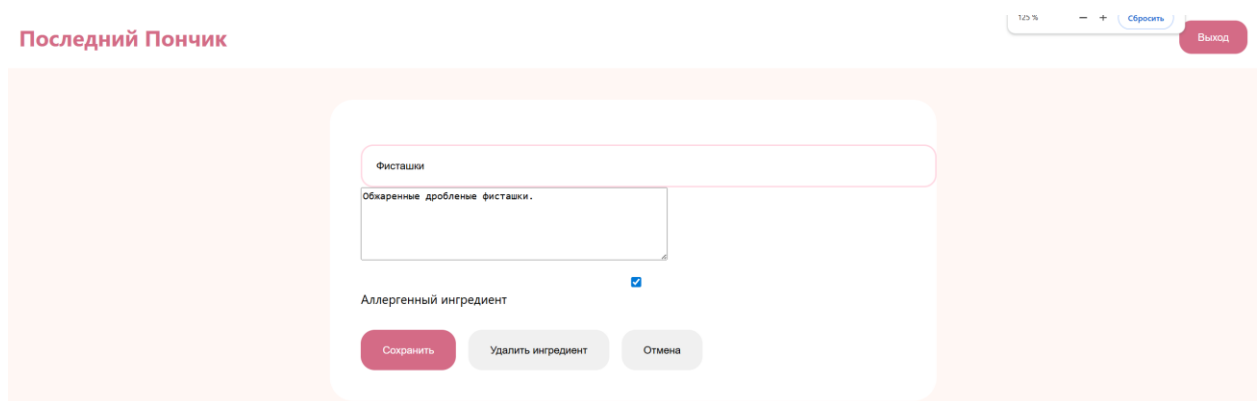


Рисунок 34 – Администратор, страница изменения ингредиентов изделий

На рисунках 35, 36, 37 показаны раздел изделий, окно создания нового изделия и окно изменения данных изделия соответственно.

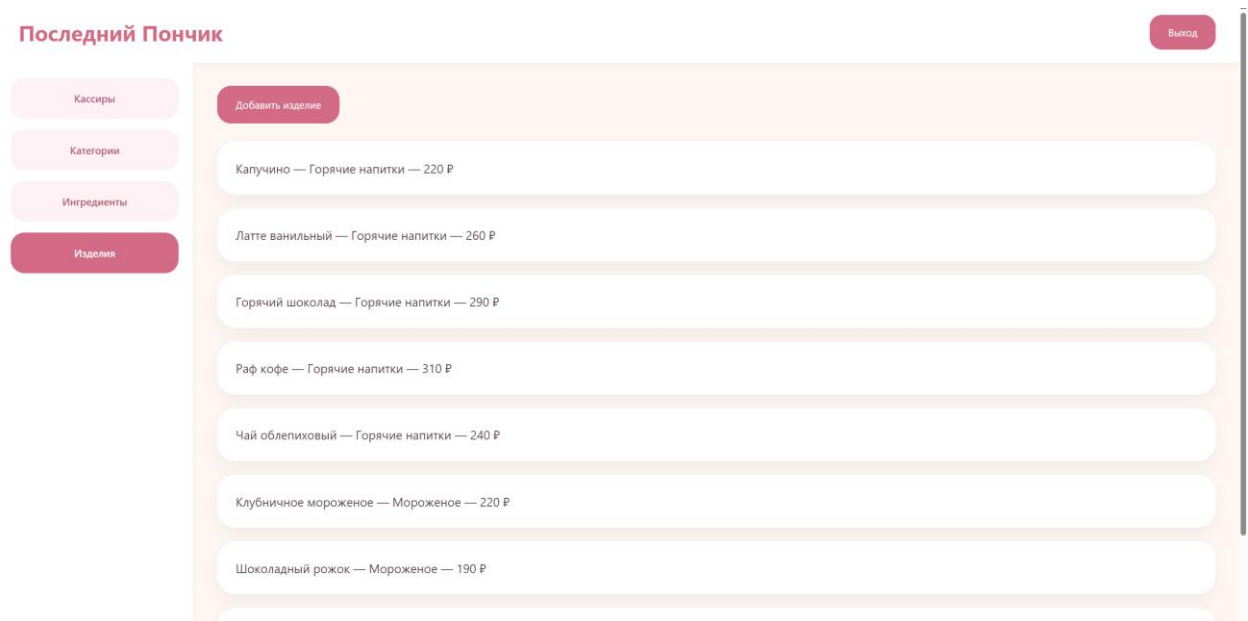


Рисунок 35 – Администратор, страница данных изделий

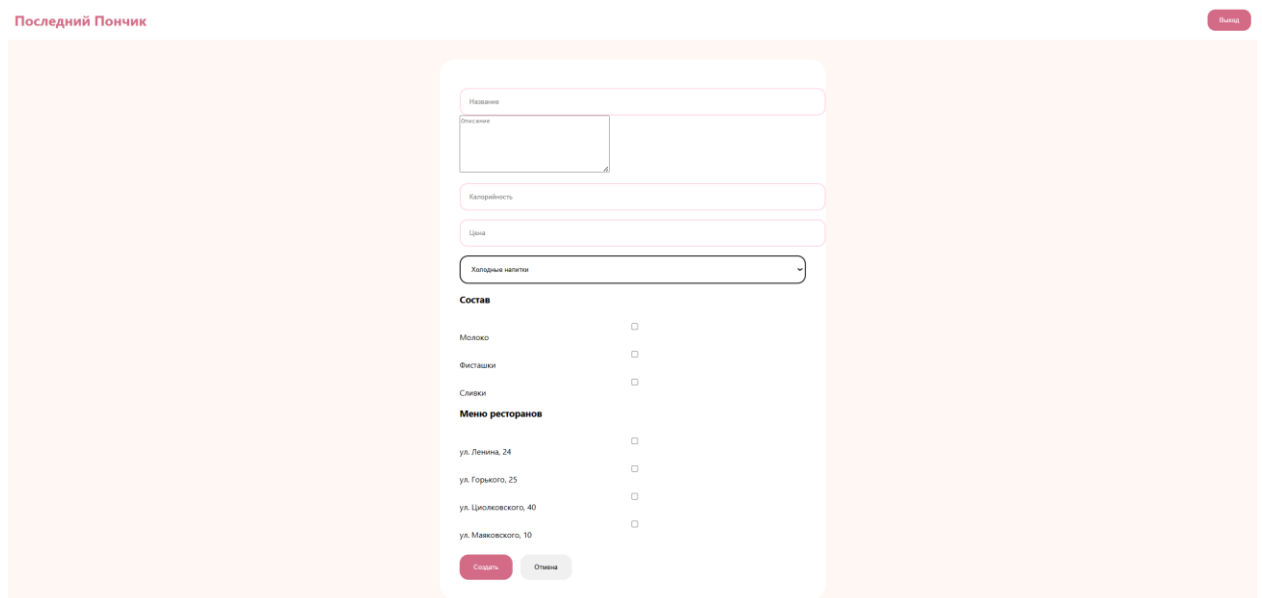


Рисунок 36 – Администратор, страница создания изделий

Название изделия

Фисташковое джелато

Описание

Нежное итальянское джелато с натуральными фисташками и сливочным вкусом.

Калорийность

340 ккал

Цена

260 Р

Категория

Мороженое

Состав

Ингредиент	Используется
Молоко	☒
Сливки	☒
Фисташки	☒
Клубничный сироп	☐
Карамель	☐

Меню ресторанов

☒ ул. Ленина, 24

☒ ул. Горького, 25

☐ ул. Циолковского, 40

☐ ул. Маяковского, 10

Сохранить

Отмена

Рисунок 37 – Администратор, страница изменения изделий

3.3 Разработка отчётов

Отчёты в данной системе должны быть сгенерированы с помощью приложения BIRT, которое позволяет настроить соединение с БД, внешний вид отчёта и его содержимое посредством различных инструментов.

В данном приложении были разработаны 2 отчёта. Первый отчет для администратора - отчета о выручке ресторанов. Второй отчет - об изменении цен на изделия. В процессе разработки были созданы источники и наборы данных. Наборы данных представлены на рисунках 38 и 39. Внешний вид отчетов – на рисунках 40-42.

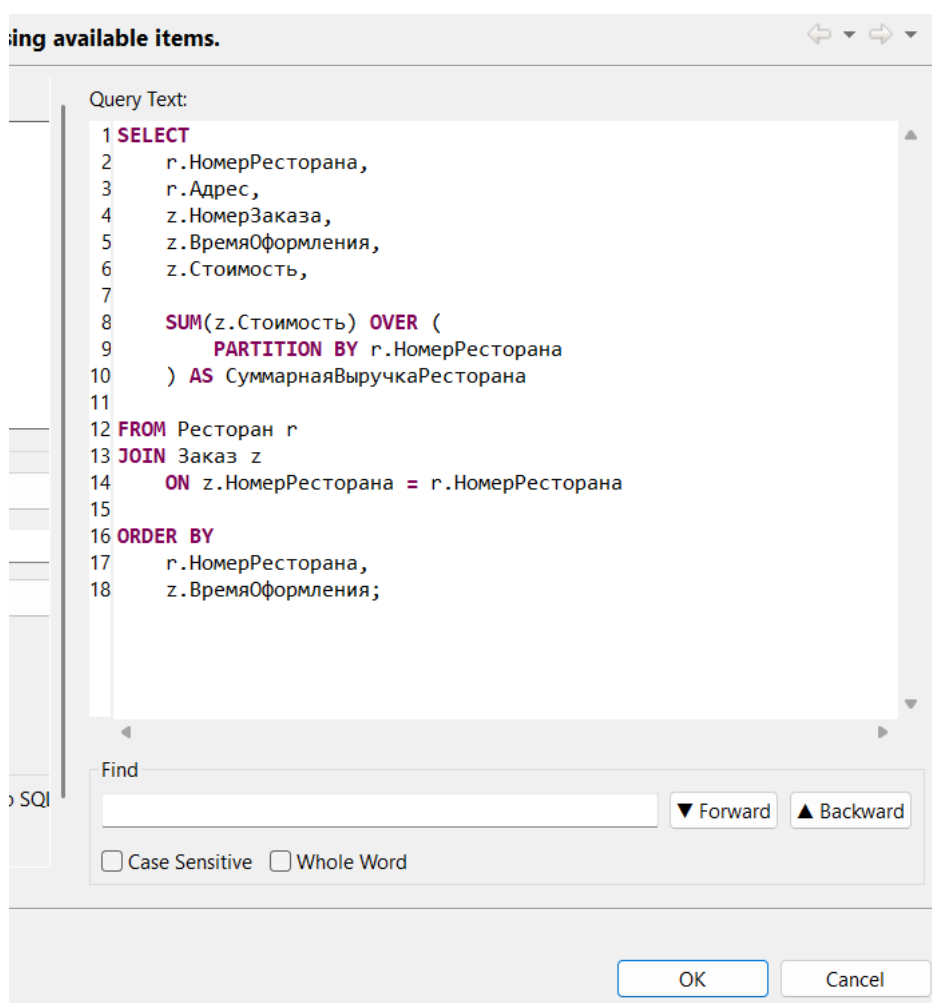


Рисунок 38 – Набор данных для отчета о выручке

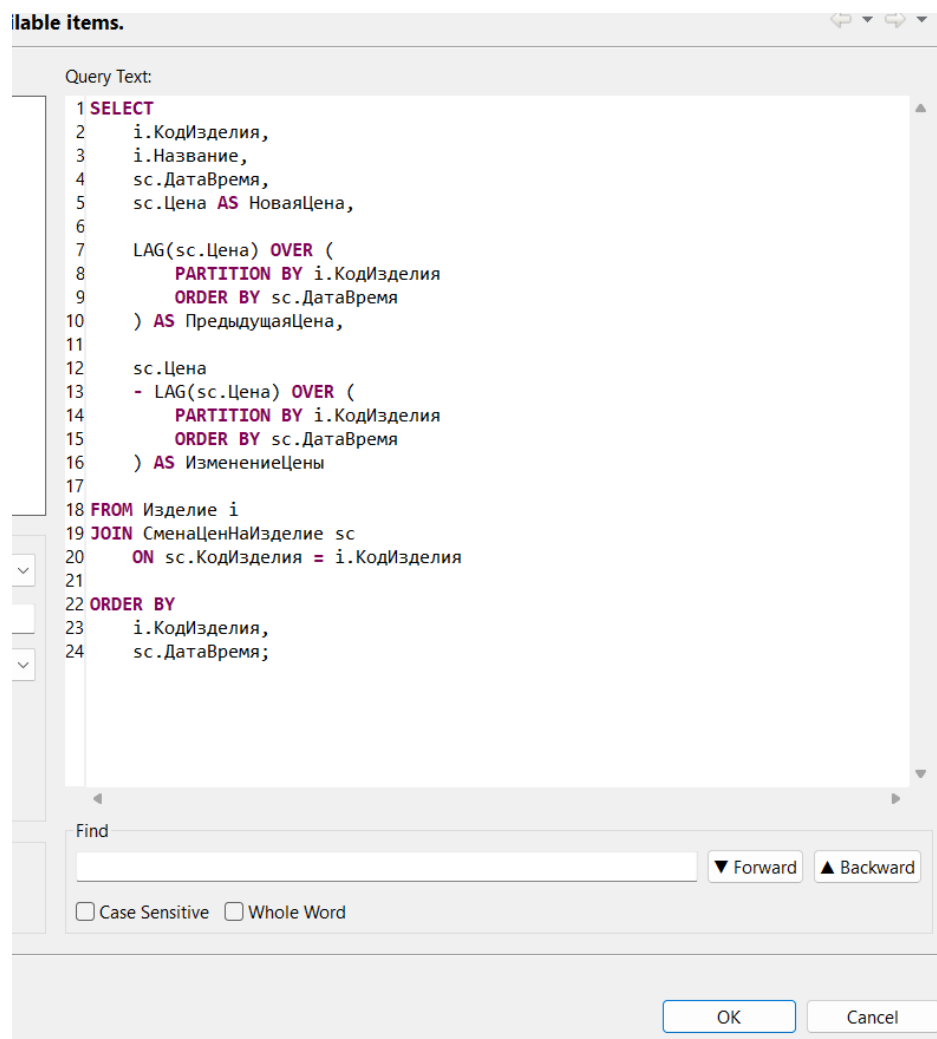


Рисунок 39 – Набор данных для отчета о ценах

НомерРесторана	Адрес	НомерЗаказа	Стоимость	ВыручкаРесторана
1	ул. Ленина, 24	1	660	10710
1	ул. Ленина, 24	2	480	10710
1	ул. Ленина, 24	4	780	10710
1	ул. Ленина, 24	10	430	10710
1	ул. Ленина, 24	11	740	10710
1	ул. Ленина, 24	16	520	10710
1	ул. Ленина, 24	17	680	10710
1	ул. Ленина, 24	22	480	10710
1	ул. Ленина, 24	23	860	10710
1	ул. Ленина, 24	28	890	10710
1	ул. Ленина, 24	29	330	10710
1	ул. Ленина, 24	33	540	10710
1	ул. Ленина, 24	36	690	10710
1	ул. Ленина, 24	39	620	10710
1	ул. Ленина, 24	42	840	10710
1	ул. Ленина, 24	45	510	10710
1	ул. Ленина, 24	50	660	10710
2	пр. Мира, 11	3	550	9950
2	пр. Мира, 11	5	430	9950
2	пр. Мира, 11	8	610	9950
2	пр. Мира, 11	12	510	9950
2	пр. Мира, 11	15	850	9950
2	пр. Мира, 11	18	440	9950
2	пр. Мира, 11	19	930	9950
2	пр. Мира, 11	24	590	9950
2	пр. Мира, 11	25	720	9950
2	пр. Мира, 11	30	640	9950
2	пр. Мира, 11	34	810	9950
2	пр. Мира, 11	35	370	9950
2	пр. Мира, 11	40	730	9950
2	пр. Мира, 11	41	560	9950
2	пр. Мира, 11	46	780	9950
2	пр. Мира, 11	47	430	9950
3	ул. Советская, 78	6	920	10200
3	ул. Советская, 78	7	340	10200
3	ул. Советская, 78	9	870	10200
3	ул. Советская, 78	13	660	10200
3	ул. Советская, 78	14	390	10200
3	ул. Советская, 78	20	310	10200
3	ул. Советская, 78	21	750	10200

Рисунок 40 – Отчет о выручке (1 страница)

НомерРесторана	Адрес	НомерЗаказа	Стоимость	ВыручкаРесторана
3	ул. Советская, 78	26	450	10200
3	ул. Советская, 78	27	580	10200
3	ул. Советская, 78	31	760	10200
3	ул. Советская, 78	32	420	10200
3	ул. Советская, 78	37	980	10200
3	ул. Советская, 78	38	450	10200
3	ул. Советская, 78	43	390	10200
3	ул. Советская, 78	44	670	10200
3	ул. Советская, 78	48	920	10200
3	ул. Советская, 78	49	340	10200
			Общая выручка	30860

Рисунок 41 – Отчет о выручке (2 страница)

	КодИзделия	Название	ДатаВремя	НоваяЦена	ПредыдущаяЦена	ИзменениеЦены
1		Клубничное мороженое	1 апр. 2026 г., 09:00	200		
1		Клубничное мороженое	16 апр. 2026 г., 08:00	220	200	20
2		Шоколадный рожок	1 апр. 2026 г., 09:00	180		
2		Шоколадный рожок	16 апр. 2026 г., 08:00	190	180	10
3		Фисташковое джелато	1 апр. 2026 г., 09:00	250		
3		Фисташковое джелато	6 апр. 2026 г., 10:00	260	250	10
4		Карамельное мороженое	1 апр. 2026 г., 09:00	220		
4		Карамельное мороженое	16 апр. 2026 г., 08:00	240	220	20
5		Капучино	1 апр. 2026 г., 09:00	210		
5		Капучино	6 апр. 2026 г., 10:00	220	210	10
6		Латте ванильный	1 апр. 2026 г., 09:00	250		
6		Латте ванильный	16 апр. 2026 г., 08:00	260	250	10
7		Горячий шоколад	1 апр. 2026 г., 09:00	280		
7		Горячий шоколад	6 апр. 2026 г., 10:00	290	280	10
8		Раф кофе	1 апр. 2026 г., 09:00	300		
8		Раф кофе	6 апр. 2026 г., 10:00	310	300	10
9		Чай облепиховый	1 апр. 2026 г., 09:00	230		
9		Чай облепиховый	16 апр. 2026 г., 08:00	240	230	10
10		Круассан	1 апр. 2026 г., 09:00	170		
10		Круассан	11 апр. 2026 г., 09:00	160	170	-10
11		Синнабон	1 апр. 2026 г., 09:00	230		
11		Синнабон	11 апр. 2026 г., 09:00	220	230	-10
12		Шоколадный маффин	1 апр. 2026 г., 09:00	180		
13		Вишневый штрудель	1 апр. 2026 г., 09:00	250		
13		Вишневый штрудель	16 апр. 2026 г., 08:00	260	250	10
14		Эклер ванильный	1 апр. 2026 г., 09:00	200		

Рисунок 42 – Отчет о ценах

Источник данных приведен на рисунке 43.

Edit Data Source - Data Source

BIRT JDBC Data Source

Connection Profile

Property Binding

Edit the selected data source.

Driver Class:

Database URL:

User Name:

Password:

JNDI URL:

Manage Drivers...

Test Connection...

Bidi Settings...

OK

Cancel

Рисунок 43 – Источник данных

3.4 Разработка сценария инсталляции клиентской программы

Данный проект был разработан с использованием фреймворка Spring, СУБД PostgreSQL и представляет из себя в большей мере серверную часть, поэтому клиенту для работы с данной ИС необходимо установить любой современный браузер и перейти на страничку сайта по заданному домену.

Для разворачивания системы необходимо инициализировать приложение Spring с некоторыми зависимостями (перечислены в приложении А), развернуть БД с использованием СУБД PostgreSQL и установить соединение BIRT с ней.

3.5 Руководство пользователя

Вход в систему. При открытии системы пользователь видит главную страницу с 4 вариантами входа: для администратора, кассира, терминала и клиента. Кассир может войти двумя способами – через раздел кассир и через раздел терминал. Администратор и кассир для входа используют номер телефона и входят в систему, подтвердив код по смс. Клиент так же может войти по номеру телефона, однако у него есть возможность не делать этого, то есть заказывать в ресторанах сети кондитерских без обязательного оформления аккаунта.

Функционал администратора. После входа в приложение администратор получает доступ к разделам управления кассирами, категориями изделий, ингредиентами и изделиями. На главной странице слева отображается список доступных разделов, а в основной области — содержимое выбранного раздела. В верхней части интерфейса располагается кнопка выхода из системы. Раздел кассиров позволяет администратору просматривать список сотрудников с указанием их паспортных данных, имени, фамилии и ресторана, в котором они работают. При необходимости администратор может добавить нового кассира с помощью кнопки «добавить кассира». В окне создания кассира указываются

имя, фамилия, паспортные данные, ресторан, номер телефона и ИНН сотрудника. В окне изменения данных кассира администратор может редактировать информацию о сотруднике и сохранять внесённые изменения.

Раздел категорий изделий предназначен для управления категориями меню. В списке отображаются существующие категории, например «холодные напитки», «горячие напитки», «мороженое», «выпечка» и другие, а также их краткие описания. С помощью кнопки «добавить категорию» администратор может создать новую категорию. В окне создания категории необходимо указать название и описание категории. В окне изменения категории администратор может изменить описание существующей категории, сохранить изменения или удалить категорию из системы. Удалить категорию возможно, только если нет ни одного изделия с такой категорией.

Раздел ингредиентов используется для хранения информации о составе изделий. В списке отображаются названия ингредиентов и их описания. При создании нового ингредиента администратор указывает его название, описание, а также отмечает наличие аллергенности с помощью флажка. В окне изменения ингредиента можно редактировать описание ингредиента, изменять отметку аллергенности или удалить ингредиент из базы данных.

Раздел изделий предназначен для управления ассортиментом продукции кондитерской. В списке изделий отображаются название изделия, его категория и цена. При создании нового изделия администратор указывает название, описание, калорийность, цену и категорию изделия, а также формирует состав продукта, выбирая необходимые ингредиенты из таблицы. Дополнительно администратор отмечает рестораны, в меню которых будет отображаться данное изделие. В окне изменения изделия администратор может изменять описание, калорийность, стоимость, категорию, состав ингредиентов и список ресторанов, в которых изделие доступно для заказа.

Функционал кассира. После авторизации кассир получает доступ к разделам «Текущие заказы» и «История заказов». На главной странице приложения отображается список активных заказов, оформленных через приложение клиента, терминал самообслуживания или непосредственно через кассу. Каждый заказ представлен в виде раскрывающегося списка, в котором в кратком виде отображаются номер заказа, время создания, номер столика или локатора (если указаны), а также текущий статус заказа. Кассир может изменять статус заказа с помощью выпадающего списка, выбирая значения «создан», «готовится», «готов» и «выдан». После изменения статуса кассир сохраняет изменения с помощью кнопки «сохранить».

При раскрытии заказа отображается подробная информация о его содержимом: список заказанных изделий, их количество и стоимость, а также итоговая сумма заказа. Дополнительно кассиру доступна история изменений заказа, где фиксируются дата, время и этапы изменения статуса. Это позволяет отслеживать процесс выполнения заказа и контролировать его готовность.

В разделе «История заказов» кассир может просматривать ранее выполненные заказы. Интерфейс данного раздела аналогичен разделу текущих заказов, однако предназначен только для просмотра завершённых заказов и их истории обработки.

Кнопка «Создать заказ» используется в случае оформления покупки непосредственно через кассу. После нажатия открывается интерфейс выбора изделий, аналогичный интерфейсу терминала самообслуживания. Кассир выбирает категорию меню, добавляет необходимые изделия в корзину и указывает их количество. При нажатии на изделие может быть открыта карточка товара с подробным описанием, составом, аллергенностью ингредиентов, калорийностью и стоимостью.

После выбора товаров кассир переходит в корзину заказа, где отображаются все добавленные позиции, их количество и итоговая стоимость.

В процессе оформления заказа кассир указывает номер телефона клиента, после чего заказ может быть подтверждён и передан в систему обработки заказов.

Функционал клиента. Клиент может пользоваться системой как без авторизации, так и после входа в аккаунт. На главной странице приложения пользователь выбирает ресторан из выпадающего списка адресов и просматривает меню, разделённое на категории. Для каждого изделия отображаются описание, цена и возможность изменить количество товара с помощью кнопок «+» и «-». При нажатии на изделие открывается карточка товара с подробной информацией: описанием, составом, указанием аллергенных ингредиентов, калорийностью и стоимостью. Неавторизованный пользователь может выполнить вход через номер телефона и СМС-подтверждение. После авторизации в шапке приложения становятся доступны разделы «Текущие заказы» и «История заказов».

При оформлении заказа пользователь переходит в корзину, где отображается список выбранных изделий, их количество и итоговая стоимость. Клиент может изменить количество товаров, удалить позиции из заказа, а также указать номер столика, если хочет получить заказ непосредственно за столиком, а не у кассы. После подтверждения заказа и оплаты информация автоматически передаётся в систему кассира для дальнейшей обработки.

В разделе «Текущие заказы» клиент может просматривать активные заказы в виде раскрывающихся списков. В кратком виде отображаются номер заказа, номер столика, стоимость и текущий статус заказа («создан», «готовится», «готов» и другие). При раскрытии заказа пользователь видит полный состав заказа с перечнем изделий, их количеством и стоимостью, а также итоговую сумму заказа.

В разделе «История заказов» отображаются завершённые заказы клиента. Для каждого заказа можно просмотреть список приобретённых изделий, итоговую стоимость, а также историю выполнения заказа с указанием даты, времени и изменения статусов на разных этапах обработки.

Заключение

В ходе выполнения работы была разработана информационная система для автоматизации работы кондитерской быстрого обслуживания «Последний Пончик». Система включает в себя приложения для клиентов, кассиров, терминалов самообслуживания и администратора, что позволяет организовать полный цикл оформления, обработки и управления заказами.

Клиентское приложение предоставляет пользователям возможность просматривать меню по категориям, получать подробную информацию об изделиях, оформлять заказы, отслеживать их текущее состояние и просматривать историю заказов. Терминал самообслуживания обеспечивает аналогичный функционал для посетителей ресторана без необходимости взаимодействия с кассиром. Приложение кассира предназначено для контроля и обработки текущих заказов, изменения их статусов, а также оформления заказов напрямую через кассу. Административное приложение обеспечивает управление кассирами, категориями изделий, ингредиентами и ассортиментом продукции.

Разработанная система обладает современным и удобным пользовательским интерфейсом, выполненным в едином визуальном стиле. Использование клиент-серверной архитектуры позволяет обеспечить централизованное хранение данных, удобное взаимодействие между различными категориями пользователей и возможность дальнейшего расширения функциональности системы.

Для проектирования базы данных были применены современные CASE-инструменты, такие как PowerDesigner и draw.io, что позволило создать нормализованную структуру, соответствующую требованиям целостности и минимизации избыточности данных. В базу данных включены ограничения, триггеры и хранимые процедуры, гарантирующие корректность хранимой

информации. SQL-скрипты разработаны с учетом всех функциональных требований системы.

Серверная часть приложения реализована на Java с использованием фреймворка Spring Boot, обеспечивающего безопасность, масштабируемость и удобное взаимодействие с базой данных PostgreSQL через JDBC. Клиентская часть представляет собой веб-интерфейс, созданный с применением HTML, CSS и JavaScript, что делает систему интуитивно понятной для пользователей.

Список используемой литературы

1. Сергеева Т.И. Базы данных: модели данных, проектирование, язык SQL: учеб. пособие / Т.И. Сергеева, М.Ю. Сергеев. Воронеж: ФГБОУ ВПО «Воронежский государственный технический университет», 2012. 233 с.
2. С. Джуба, А. Волков – «Изучаем PostgreSQL 10»: учеб. пособие / С. Джуба, А. Волков. — СПб.: БХВ-Петербург, 2018. — 336 с.: ил.;
3. Крейг Уоллс – «Spring в действии»: учеб. пособие, «ДМК-Пресс», 2022 год, 544 с.
4. Diana Peh. BIRT: A Field Guide to Reporting / Diana Peh, Nola Hague, Jane Tatchell. – Boston: MA, 2008. – P. 794;
5. Роберт Мартин – «Чистый код», Издательство «Питер», 2018 год, 464 с.